



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

**Diseño e implementación de un modelo de mantenimiento asistido
por inteligencia artificial para una arquitectura de microservicios en
el sistema PortalAsig**

Trabajo Especial de Grado presentado ante la ilustre
Universidad Central de Venezuela por:
Br. Frank Ángel Ponte Delgado
Tutor: Prof. Walter Hernández

Agradecimientos

[Placeholder: Escribir aquí los agradecimientos]

Resumen

El sistema PortalAsig opera desde el año 2016 sobre una arquitectura monolítica que acumuló seis años de deuda técnica sin proceso sistemático de mantenimiento, en un contexto sin equipo de desarrollo de tiempo completo y con colaboradores académicos temporales. Esta situación generó un grado de acoplamiento que dificultaba cualquier intervención y hacía depender el sistema del conocimiento tácito de cada mantenedor.

El presente trabajo documenta el rediseño del backend hacia una arquitectura de microservicios que descompone el sistema en dominios acotados e independientes —UAA (gestión de identidad), Site (sitios académicos) y Notify (notificaciones)— cada uno con base de datos propia y ciclo de despliegue autónomo. El frontend heredado fue adaptado para operar como cliente distribuido mediante Vuex modular, guards de navegación, autenticación OAuth2/JWT y clientes Axios multi-servicio. Sobre este ecosistema se implementaron técnicas sistemáticas de aseguramiento de calidad que abarcan todo el ciclo de vida del código —desde el análisis estático y la integración continua hasta las pruebas end-to-end y la medición de cobertura— y se instrumentó observabilidad distribuida para consolidar métricas, logs y trazas en un panel centralizado.

Sobre esta infraestructura se construyó el servicio PAIMA (PortalAsig Intelligent MAPE-K), que ejecuta un ciclo de autogestión del mantenimiento donde las fases de Análisis y Planificación se delegan a un modelo de inteligencia artificial. PAIMA monitorea el ecosistema, detecta anomalías, clasifica incidencias y propone acciones de remediación que el operador humano evalúa y aprueba.

La validación mediante pruebas de integración, validación end-to-end y simulación de caos demostró que el ecosistema aísla fallos, mantiene métricas objetivas de calidad y reduce la carga cognitiva del mantenedor. El resultado es un sistema desacoplado, observable y autogestionado que puede ser mantenido por colaboradores académicos temporales sin depender del conocimiento tácito.

Palabras clave: arquitectura de microservicios, mantenimiento de software, integración continua, observabilidad distribuida, computación autónoma, MAPE-K, inteligencia artificial, autogestión del mantenimiento.

Índice general

Agradecimientos	1
Resumen	2
Introducción	8
1. Planteamiento del problema	11
1.1. Justificación	12
1.2. Objetivo General	13
1.3. Objetivos Específicos	13
2. Marco Teórico	15
2.1. Evolución de los Paradigmas Web	15
2.1.1. De la Web 1.0 a la Web 3.0	15
2.1.2. Web 3.0: Distribución, Autonomía e Inteligencia Artificial	16
2.2. Arquitecturas de Software	17
2.2.1. Arquitecturas Monolíticas	17
2.2.2. Arquitecturas Basadas en Microservicios	17
2.2.3. Comparación entre Arquitecturas Monolíticas y Microser-	
vicios	18
2.3. Tecnologías para el Desarrollo y Operación de Microservicios . .	18
2.3.1. Contenedores y Orquestación	19
2.3.2. Framework y Construcción	19
2.3.3. Seguridad y Comunicación	20
2.3.4. Persistencia	21
2.3.5. Observabilidad Distribuida	22
2.3.6. Calidad de Software y Testing	23
2.3.7. Documentación y Frontend	25
2.3.8. Modelos de Lenguaje de Gran Escala y Locales	25
2.4. Mantenimiento de Software	26
2.4.1. Tipos de Mantenimiento	26
2.4.2. Retos en Sistemas Distribuidos	26
2.4.3. Prácticas Modernas de Aseguramiento de Calidad	27
2.5. Computación Autónoma	27

2.5.1.	Concepto y Propiedades Self-CHOP	27
2.6.	Modelo MAPE-K	28
2.6.1.	Componentes del Ciclo	28
2.6.2.	De MAPE-K a la Integración de Inteligencia Artificial	28
2.7.	Sistemas Auto-adaptativos	29
2.8.	Inteligencia Artificial aplicada al Mantenimiento de Software	30
2.9.	AIOps y Automatización de Operaciones	30
2.10.	Integración de IA en Arquitecturas de Microservicios	30
3.	Marco Aplicativo	32
3.1.	Migración del Backend a una Arquitectura de Microservicios	32
3.1.1.	Análisis del Sistema Monolítico Legacy	32
3.1.2.	Diseño de la Arquitectura de Microservicios	35
3.1.3.	Stack Tecnológico del Backend	36
3.1.4.	Técnicas de Mantenimiento de Software Implementadas	39
3.1.5.	Despliegue, Gestión y Migración de Datos	42
3.1.6.	Comparativa Cualitativa: Monolito vs. Microservicios	42
3.2.	Servicio PAIMA — Autogestión del Mantenimiento Asistido por IA	44
3.2.1.	Motivación y Contexto	44
3.2.2.	Arquitectura Interna de PAIMA	45
3.2.3.	Fase Monitor: Recolección de Datos Operativos	46
3.2.4.	Fase Analyze y Plan: Asistencia con Inteligencia Artificial	47
3.2.5.	Fase Execute: Automatización y Notificación	47
3.2.6.	Fase Knowledge: Registro de Incidentes y Feedback	47
3.2.7.	Interfaz de Gestión de Incidentes (Frontend)	48
3.3.	Adaptación del Frontend a la Arquitectura Distribuida	49
3.3.1.	Diagnóstico del Frontend Legacy	52
3.3.2.	Reestructuración Arquitectónica	52
3.3.3.	Migración de Autenticación	53
3.3.4.	Integración con los Microservicios	53
4.	Pruebas y Resultados	55
4.1.	Metodología y Entorno de Pruebas	55
4.2.	Pruebas de Integración y Cobertura de Código	56
4.2.1.	Estrategia de Pruebas de Integración	56
4.2.2.	Cobertura de Código con JaCoCo	56
4.3.	Pruebas End-to-End	58
4.3.1.	Selección de Herramienta y Diseño de Flujos	58
4.3.2.	Resultados de Ejecución	59
4.3.3.	Validación de la Arquitectura Frontend Adaptada	60
4.4.	Benchmarking de Rendimiento	65
4.4.1.	Diseño del Experimento	65
4.4.2.	Análisis de Diseño Experimental	66
4.5.	Simulación de Caos y Recuperación Asistida	67
4.5.1.	Diseño del Escenario	67
4.5.2.	Comportamiento Observado	69

4.5.3. Lecciones del Experimento	69
4.6. Análisis Comparativo Global	73
5. Conclusiones y Recomendaciones	76
5.1. Conclusiones	76
5.2. Recomendaciones y Trabajo Futuro	77

Índice de figuras

3.1. Diagrama entidad-relación simplificado del modelo de datos monolítico, mostrando la tabla <code>site</code> como núcleo con sus 15+ relaciones hacia entidades dependientes.	34
3.2. Arquitectura del ecosistema de microservicios de PortalAsig, mostrando el proxy Nginx, el Config Server, los cuatro microservicios (UAA, Site, Notify, PAIMA), sus bases de datos MySQL aisladas, el proveedor LLM externo, el stack de observabilidad (Prometheus, Loki, Grafana) y las conexiones de monitoreo de PAIMA hacia los endpoints <code>/actuator/health</code> de cada microservicio, todo desplegado sobre la red virtual de Docker Compose.	36
3.3. Estructura de directorios de migraciones Flyway en un microservicio.	39
3.4. Diagrama de herencia de dependencias Maven.	40
3.5. Diagrama de clases de PAIMA mostrando los paquetes del ciclo MAPE-K.	46
3.6. Screenshot del panel de control de PAIMA.	48
3.7. Listado de alertas autonómicas priorizadas visibles para el administrador.	49
3.8. Screenshot de la vista de detalle de un incidente P1 en PAIMA.	50
3.9. Interacción del administrador con el agente de IA para refinar el diagnóstico antes de ejecutar una acción sobre la alerta.	51
4.1. Reporte de cobertura de código (JaCoCo) enfocado en los módulos críticos del orquestador autonómico (MAPE).	58
4.2. Ejecución exitosa de las pruebas End-to-End (E2E) mediante Cypress.	59
4.3. Secuencia de pasos del flujo de autenticación OAuth2/JWT validada mediante Cypress: obtención de token anónimo, login con credenciales, persistencia en cookies y refresco automático.	61
4.4. Validación de guards de navegación: intento de acceso a vista administrativa con rol de estudiante (izquierda) y redirección automática a <code>/home</code> (derecha).	62
4.5. Inspección de red durante flujo de creación de sitio: peticiones a UAA (autenticación), Site (contenido) y PAIMA (alertas) desde instancias Axios independientes.	63

4.6. Estado del store Vuex modular durante flujo de selección de asignatura: el módulo course se actualiza sin modificar site ni session	64
4.7. Dashboard de Grafana proyectado para la visualización de métricas durante la ejecución de benchmarks con k6.	68
4.8. Portal de asignaturas en estado degradado tras la caída del servicio Site. El usuario mantiene la sesión activa (autenticación gestionada por UAA), pero el listado de sitios académicos muestra un indicador de error en lugar de bloquear toda la interfaz. . . .	70
4.9. Portal de asignaturas tras la recuperación del servicio Site. El listado de sitios académicos carga correctamente, confirmando que el reinicio ejecutado por PAIMA restauró la funcionalidad completa.	71
4.10. Notificación de alerta crítica generada por el orquestador autónómico.	72
4.11. Alerta crítica visible en el portal administrativo con el asistente IA integrado.	72
4.12. Aprobación y ejecución de la acción de auto-reparación mediante el asistente IA.	73

Introducción

El sistema PortalAsig fue desarrollado en el año 2016 para la gestión académica de la Facultad de Ciencias de la Universidad Central de Venezuela. Inicialmente concebido como una plataforma centralizada que facilitara la comunicación entre docentes y estudiantes, el sistema ha evolucionado durante más de una década mediante modificaciones incrementales que expandieron su funcionalidad sin alterar su arquitectura fundamental. El frontend del sistema fue rediseñado en el año 2020 por Vásquez Balza [1], quien migró la interfaz de usuario desde Angular hacia Vue.js bajo un enfoque orientado a la mantenibilidad. Este rediseño mejoró sustancialmente la organización del código cliente, sentó las bases para una evolución controlada de la capa de presentación y demostró que un rediseño arquitectónico bien planificado puede extender la vida útil de un sistema sin reescribirlo por completo. No obstante, el backend del sistema permaneció basado en una arquitectura monolítica que acumuló, durante seis años consecutivos, una deuda técnica creciente sin proceso sistemático de mantenimiento.

El backend de PortalAsig permanece tecnológicamente estancado en una arquitectura monolítica que concentra todas las responsabilidades del negocio —autenticación de usuarios, gestión de asignaturas, creación de sitios académicos, administración de evaluaciones, foros, entregas de archivos, envío de correos y encuestas— dentro de un único proceso desplegable. La ausencia de personal de desarrollo de tiempo completo dedicado al sistema impide actualizar esta arquitectura: los colaboradores académicos temporales que asumen el mantenimiento deben operar sobre una base de código de difícil comprensión, sin documentación exhaustiva ni procesos formales de transferencia de conocimiento. A medida que el sistema creció en funcionalidad, la base de código se densificó progresivamente, generando acoplamiento entre dominios que originalmente eran independientes. La ausencia de pruebas automatizadas, de mecanismos de validación de cambios y de herramientas de observabilidad convirtió cada modificación en un ejercicio de alto riesgo, donde la introducción de un cambio local podía afectar funcionalidades aparentemente no relacionadas.

El contexto operativo agrava esta problemática. El sistema carece de un equipo de desarrollo y operaciones de tiempo completo; el soporte técnico recurrente proviene del grupo docente y las mejoras se originan en colaboradores académicos temporales —estudiantes de Trabajo Especial de Grado, de seminario o de servicio comunitario— que permanecen en el proyecto entre uno y dos semestres. Esta rotación perpetua —un fenómeno documentado en la literatura

de ingeniería de software como *job rotation* [2]— implica que el conocimiento tácito acumulado sobre el código, la infraestructura y los procesos de despliegue se pierde cada vez que un colaborador concluye su período, obligando al siguiente a reconstruirlo, al menos parcialmente, sobre una base de código de difícil comprensión. La infraestructura donde se aloja el monolito opera bajo una modalidad *on-premise*, sin acceso a servicios de computación en la nube ni a un equipo de DevOps dedicado. El despliegue se realiza de forma manual mediante conexión directa al servidor. En estas condiciones, cualquier herramienta adicional que requiera infraestructura propia, configuración compleja o mantenimiento continuo constituye un pasivo técnico que el siguiente colaborador deberá asumir sin garantía de transferencia de conocimiento.

Frente a esta situación, se diseñaron e implementaron dos líneas de transformación. La primera consistió en el rediseño del backend hacia una arquitectura de microservicios que descompone el sistema en dominios acotados e independientes—UAA (gestión de identidad), Site (sitios académicos) y Notify (notificaciones)—cada uno con su propia base de datos, su esquema de versionado y su ciclo de despliegue autónomo. Esta transformación elimina el acoplamiento referencial del modelo monolítico y habilita el aislamiento de fallos, de modo que la caída de un servicio no compromete la estabilidad del ecosistema completo. Sin embargo, la migración a microservicios introduce una complejidad operativa inherente a los sistemas distribuidos: múltiples procesos, múltiples bases de datos, latencia de red, tolerancia a fallos y monitoreo distribuido.

La segunda línea de transformación adaptó el frontend heredado para operar como cliente de esa nueva arquitectura distribuida: se reestructuró el store Vuex en módulos por dominio, se implementaron guards de navegación con siete niveles de permisos, se reemplazó la autenticación propietaria por un flujo OAuth2/JWT con cookies y refresh token, y se configuraron múltiples instancias de Axios que reflejan la topología de los microservicios (UAA, Site, Notify).

Para afrontar la complejidad operativa del ecosistema distribuido, se implementaron técnicas sistemáticas de mantenimiento de software que abarcan todo el ciclo de vida del código. Se incorporaron mecanismos de validación automatizada que analizan el estilo del código antes de su fusión, ejecutan pruebas sobre bases de datos reales y miden la cobertura de las mismas. Se diseñaron pruebas que simulan flujos completos del usuario final sobre la interfaz, verificando que las capas de presentación, aplicación y persistencia operen de forma coordinada. Asimismo, se instrumentaron mecanismos de observabilidad distribuida que recolectan métricas de rendimiento, agregan registros de ejecución y visualizan correlaciones entre eventos. Estas técnicas no son complementos optativos: son la condición previa que permite a un colaborador académico temporal comprender el estado del sistema, detectar anomalías antes de que afecten a los usuarios y realizar cambios con confianza, reduciendo la dependencia del conocimiento tácito.

Sobre esta infraestructura se construyó el servicio PAIMA (PortalAsig Intelligent MAPE-K), que ejecuta un ciclo de autogestión del mantenimiento donde las fases de Análisis y Planificación se delegan a un modelo de inteligencia artificial. PAIMA monitorea el ecosistema, detecta anomalías, clasifica

incidencias y propone acciones de remediación que el operador humano evalúa y aprueba. La validación mediante pruebas de integración, validación end-to-end y simulación de caos demostró que el ecosistema aísla fallos, mantiene métricas objetivas de calidad y reduce la carga cognitiva del mantenedor. El resultado es un sistema desacoplado, observable y autogestionado que puede ser mantenido por colaboradores académicos temporales sin depender del conocimiento tácito.

El presente documento se organiza en cinco capítulos. El Capítulo 1 plantea el problema, justifica la necesidad de transformación y establece los objetivos general y específicos del trabajo. El Capítulo 2 presenta el marco teórico, abordando las arquitecturas monolíticas y de microservicios, el modelo MAPE-K, la computación autonómica, la observabilidad en sistemas distribuidos y las tendencias convergentes que sustentan la autogestión del mantenimiento asistida por inteligencia artificial. El Capítulo 3 documenta el desarrollo aplicativo: el análisis del monolito legacy, el diseño de la arquitectura de microservicios, las técnicas de mantenimiento implementadas y la arquitectura del servicio PAIMA. El Capítulo 4 describe la estrategia de pruebas diseñada, los escenarios ejecutados y los resultados obtenidos en cobertura de código, validación end-to-end, simulación de caos y análisis comparativo. El Capítulo 5 cierra el trabajo con las conclusiones, las limitaciones observadas y las líneas de evolución propuestas para el ecosistema.

Capítulo 1

Planteamiento del problema

El sistema PortalAsig, desarrollado para la gestión académica en la Facultad de Ciencias de la Universidad Central de Venezuela, se ejecuta desde el año 2016 sobre una arquitectura monolítica que concentra todas sus funcionalidades —autenticación de usuarios, gestión de asignaturas, sitios académicos, evaluaciones, foros, entregas de archivos, envío de correos y encuestas— en un único proceso Node.js. En este backend, el archivo `site.js`, ubicado en el directorio `api/controllers/`, alcanza 3.629 líneas de código que gestionan quince dominios de negocio distintos, desde bibliografías hasta calificaciones, sin mecanismo de aislamiento que impida que un fallo en un módulo comprometa la estabilidad de la aplicación completa. La entidad `Site`, definida en el modelo `api/models/site.js`, declara dieciocho relaciones con otras tablas de la base de datos PostgreSQL, de modo que su eliminación requiere la ejecución manual de dieciocho operaciones de borrado en cascada dentro de un hook del modelo, implementado mediante promesas anidadas que, si fallan, dejan el sistema en un estado de inconsistencia parcial.

Esta acumulación de deuda técnica durante seis años de modificaciones incrementales sin un proceso sistemático de mantenimiento ha generado un grado de acoplamiento que dificulta cualquier intervención por parte de colaboradores académicos temporales. El monolito no cuenta con un directorio de pruebas automatizadas, ni pipelines de integración continua, ni herramientas de análisis estático de código. El despliegue en producción se realiza de forma manual directamente sobre el servidor del Centro de Computación de la Facultad de Ciencias, sin mecanismos que validen el comportamiento del sistema después de cada cambio. La autenticación emplea un mecanismo propietario basado en cadenas aleatorias almacenadas en una tabla `Session` de PostgreSQL, sin soporte para tokens JWT ni protocolos OAuth2, lo que impide la escalabilidad horizontal y complica la integración con estándares de seguridad modernos. El sistema no expone endpoints de métricas ni cuenta con agregación centralizada de logs; la única forma de detectar una anomalía es la observación directa de fallas visibles por parte de usuarios o mantenedores.

La respuesta ante un error del sistema es, por tanto, empírica y reactiva,

especialmente durante los semestres académicos en curso, cuando el pico de uso expone con mayor frecuencia las fallas acumuladas en la base de código. Este mecanismo de mantenimiento depende en gran medida de acciones manuales y del conocimiento implícito de los colaboradores, sin contar con mecanismos formales de monitoreo, observabilidad o diagnóstico que permitan gestionar el sistema de forma eficiente. Como consecuencia, se dificulta la detección temprana de problemas, se incrementan los tiempos de resolución de fallas y se limita la incorporación de nuevas características.

En este escenario se evidencia la necesidad de transformar tanto la arquitectura como el proceso de mantenimiento de PortalAsig. La migración hacia una arquitectura de microservicios resuelve el problema del acoplamiento arquitectónico, descomponiendo el sistema en dominios acotados con bases de datos independientes y ciclos de despliegue autónomos. No obstante, esta transformación introduce una complejidad operativa inherente a los sistemas distribuidos: múltiples procesos, múltiples bases de datos, latencia de red, tolerancia a fallos y monitoreo distribuido. En un entorno donde no existe un equipo de operaciones dedicado y donde el mantenimiento recae en colaboradores académicos temporales que permanecen uno o dos semestres, esta complejidad puede convertirse en un cuello de botella crítico si no se acompaña de mecanismos automatizados de validación, observabilidad y autogestión.

Frente a esta situación, se identifica la necesidad de un enfoque que combine la arquitectura modular con mecanismos de observabilidad distribuida y técnicas de inteligencia artificial, que permitan asistir el mantenimiento del sistema y reducir la carga cognitiva sobre el mantenedor humano. Este enfoque posibilita el paso de un esquema enteramente reactivo a uno sistemático, donde el propio ecosistema detecta anomalías, diagnostica causas y propone acciones de remediación que un operador evalúa y aprueba.

1.1. Justificación

La modernización del sistema PortalAsig responde a una realidad operativa concreta: el sistema carece de un equipo de desarrollo y operaciones de tiempo completo asignado. El único soporte técnico recurrente proviene del grupo docente, y las mejoras al sistema se originan exclusivamente en colaboradores académicos temporales —estudiantes de Trabajo Especial de Grado, de seminario o de servicio comunitario— que permanecen en el proyecto entre uno y dos semestres. Esta rotación perpetua implica que el conocimiento tácito acumulado sobre el código, la infraestructura y los procesos de despliegue se pierde cada vez que un colaborador concluye su período, obligando al siguiente a reconstruir ese conocimiento, al menos parcialmente, sobre una base de código de difícil comprensión.

El monolito de producción de PortalAsig se aloja en los servidores del Centro de Computación de la Facultad de Ciencias, bajo una modalidad *on-premise* donde no existe acceso a servicios de computación en la nube ni equipo de DevOps dedicado. El despliegue se realiza de forma manual mediante conexión

directa al servidor. En estas condiciones, cualquier herramienta adicional que requiera infraestructura propia, configuración compleja o mantenimiento continuo constituye un pasivo técnico que el siguiente colaborador deberá asumir sin garantía de transferencia de conocimiento.

La adopción de una arquitectura basada en microservicios permite descomponer el sistema en dominios independientes, cada uno comprensible de forma aislada y con un área de impacto acotada ante modificaciones. Sin embargo, esta transformación incrementa la complejidad operativa: un ecosistema distribuido tiene múltiples puntos de fallo, cada servicio expone sus propias métricas y logs, y la correlación de problemas entre nodos requiere observar simultáneamente múltiples fuentes de datos. Sin operadores fijos que conozcan la arquitectura en profundidad, esta complejidad puede superar los beneficios arquitectónicos si no se acompaña de automatización.

En este contexto, la incorporación de un mecanismo de autogestión del mantenimiento asistido por inteligencia artificial representa una alternativa viable para automatizar tareas de análisis, detección de anomalías y apoyo a la toma de decisiones. Este enfoque permite reducir la dependencia de intervenciones manuales y mejorar la capacidad de respuesta ante fallas, siendo especialmente relevante en un sistema donde los recursos humanos son limitados, temporales y rotativos. El resultado no es un sistema que elimine al mantenedor, sino uno que reduce su carga cognitiva y acelere sus decisiones mediante diagnósticos consolidados y recomendaciones estructuradas.

1.2. Objetivo General

Diseñar e implementar una arquitectura de microservicios para el backend del sistema PortalAsig, adaptar el frontend heredado para operar como cliente de dicha arquitectura distribuida e incorporar técnicas sistemáticas de mantenimiento de software —integración continua con validación automatizada, análisis estático de código, pruebas de integración sobre bases de datos reales, pruebas end-to-end e instrumentación de observabilidad distribuida— que mejoren su mantenibilidad, capacidad de adaptación y sostenibilidad en el tiempo; e integrar, sobre dicho ecosistema, el servicio PAIMA como mecanismo de autogestión del mantenimiento asistido por inteligencia artificial mediante el ciclo MAPE-K.

1.3. Objetivos Específicos

- Documentar el estado actual del sistema PortalAsig y caracterizar sus carencias estructurales mediante análisis de código fuente, modelo de datos y procesos de despliegue.
- Diseñar una arquitectura de microservicios que desacople el backend en dominios independientes (UAA, Site, Notify), cada uno con su propia base de datos, su esquema de versionado y su ciclo de despliegue autónomo.

- Adaptar el frontend heredado para operar como cliente de la nueva arquitectura distribuida, reestructurando el estado por dominio, migrando la autenticación a estándares OAuth2/JWT e implementando mecanismos de comunicación con los microservicios backend.
- Implementar técnicas sistemáticas de aseguramiento de calidad que abarquen análisis estático, pruebas de integración sobre bases de datos reales, medición de cobertura de código, validación end-to-end de la interfaz y automatización del pipeline de integración continua.
- Diseñar e implementar el servicio PAIMA, que ejecute un ciclo MAPE-K donde las fases de Análisis y Planificación se deleguen a un modelo de inteligencia artificial, configurado con proveedores intercambiables y fallback determinista.
- Instrumentar mecanismos de observabilidad distribuida para la recolección de métricas, agregación de logs y visualización centralizada, que constituyan la base sensorial sobre la cual PAIMA opere.
- Evaluar el ecosistema resultante mediante pruebas de integración, validación end-to-end, simulación de caos con PAIMA y análisis comparativo cualitativo frente al monolito legacy.

Capítulo 2

Marco Teórico

2.1. Evolución de los Paradigmas Web

2.1.1. De la Web 1.0 a la Web 3.0

La World Wide Web ha atravesado transformaciones estructurales que definen no solo cómo se consume información, sino cómo se construyen y operan los sistemas de software. La Web 1.0, predominante durante la década de 1990, operaba bajo un modelo de lectura pasiva: páginas estáticas, contenido generado exclusivamente por administradores y una arquitectura cliente-servidor centralizada donde el servidor era el único nodo productor de información. El usuario era un consumidor sin capacidad de modificación.

La transición hacia la Web 2.0, consolidada a partir de 2004, introdujo la participación activa del usuario. Plataformas sociales, wikis, blogs y sistemas de gestión de contenido transformaron la web en un espacio colaborativo donde los usuarios generan datos, establecen relaciones y modifican el estado de las aplicaciones en tiempo real. Sin embargo, esta evolución incrementó drásticamente la complejidad de los sistemas backend: arquitecturas monolíticas que originalmente atendían flujos simples de lectura tuvieron que adaptarse para procesar escrituras concurrentes, relaciones sociales, notificaciones en tiempo real y volúmenes masivos de datos generados por los usuarios. La acumulación de funcionalidades sobre bases monolíticas originó, en muchos casos, sistemas de alta complejidad y baja mantenibilidad.

El término Web 3.0 fue acuñado originalmente por Berners-Lee, Hendler y Lassila para describir la Web Semántica, una visión donde los datos estarían estructurados de forma que las máquinas pudieran interpretarlos y razonar sobre ellos de manera autónoma [3]. Durante los años siguientes, el ecosistema de las criptomonedas adoptó la expresión para referirse a una internet descentralizada basada en tecnologías de contabilidad distribuida (blockchain), asociándose con la descentralización económica y la propiedad de datos por parte de los usuarios [4]. Si bien esta acepción ha dominado el discurso popular e industrial durante la década de 2010, la literatura reciente ha reinterpretado el paradigma

desde la perspectiva de la ingeniería de software. Shen et al. [5] proponen un framework arquitectónico para Web 3.0 donde la inteligencia artificial resuelve desafíos fundamentales en cada capa del ecosistema, mientras que Kuai et al. [6] introducen el concepto de Web Intelligence 3.0 como materialización de una sociedad inteligente interconectada. Este trabajo adopta esta última acepción, reconociendo la Web 3.0 como un paradigma de sistemas distribuidos, autónomos e inteligentes, donde la inteligencia artificial, los agentes autónomos y la arquitectura de microservicios constituyen su infraestructura técnica fundamental, más allá de las connotaciones exclusivamente financieras de la tecnología de contabilidad distribuida.

2.1.2. Web 3.0: Distribución, Autonomía e Inteligencia Artificial

La literatura reciente reinterpreta el paradigma de la Web 3.0 desde la perspectiva de la ingeniería de software como un espacio donde convergen tres tendencias tecnológicas, cada una con raíces en disciplinas distintas pero complementarias entre sí.

Distribución arquitectónica. Tanto la acepción blockchain de Web 3.0 como la propuesta de Web Intelligence 3.0 [6] comparten el principio de que los sistemas deben dejar de depender de un único proceso monolítico para descomponerse en componentes independientes que operan sobre una red. Esta propiedad no implica necesariamente una red pública de blockchain, sino una red interna de servicios interconectados mediante protocolos estándar, típicamente HTTP/REST o mensajería asíncrona. La distribución habilita la evolución continua del software sin interrumpir la operación global y constituye la base material sobre la cual se construyen las demás capacidades.

Observabilidad y auto-gestión. La complejidad operativa inherente a los sistemas distribuidos exige mecanismos de monitoreo, agregación de telemetría y toma de decisiones automatizada. Esta necesidad, heredada de la computación autónoma (Sección 2.5), transforma el mantenimiento de software de un proceso reactivo y manual a un ciclo continuo donde el sistema monitorea su propio estado, detecta anomalías y propone acciones de corrección. La observabilidad no es un complemento optativo: es la condición previa que permite a los mecanismos de control razonar sobre el estado interno del ecosistema.

Inteligencia artificial como asistente operativo. La inteligencia artificial ha comenzado a desempeñar un papel transversal en la infraestructura de Web 3.0 [5], resolviendo desafíos que van desde la gestión de datos hasta la gobernanza del ecosistema. En particular, los modelos de lenguaje de gran escala (LLMs) pueden interpretar telemetría operativa, sintetizar diagnósticos en lenguaje natural y proponer planes de remediación que un operador humano evalúa y aprueba. En esta acepción, la IA no sustituye al administrador: actúa como un asistente que reduce la carga cognitiva y acelera la toma de decisiones.

Es importante aclarar que estas tres tendencias no constituyen una definición normativa de Web 3.0, sino una convergencia observable en la literatura técnica reciente. Para que un sistema heredado pueda incorporar estas capacidades,

su arquitectura subyacente debe permitir la distribución de componentes, la exposición de telemetría y la integración de módulos inteligentes. Un monolito acoplado, sin instrumentación ni puntos de extensión, constituye una barrera estructural que impide esta evolución.

2.2. Arquitecturas de Software

2.2.1. Arquitecturas Monolíticas

El desarrollo de software tradicional frecuentemente se orienta hacia la construcción de aplicaciones monolíticas. En este tipo de arquitectura, todos los componentes del sistema —interfaz de usuario, lógica de negocio y acceso a datos— se empaquetan y despliegan como una única unidad lógica y ejecutable [7]. Si bien este enfoque simplifica el desarrollo inicial, las pruebas y el despliegue a pequeña escala, tiende a presentar problemas significativos de escalabilidad y mantenimiento a medida que la base de código crece.

Un fenómeno recurrente en sistemas monolíticos de larga vida es la *deuda técnica*: el conjunto de compromisos de diseño y codificación que aceleran la entrega en el corto plazo a costa de dificultar la evolución futura. La deuda técnica se manifiesta en controladores de dimensiones críticas, modelos de datos con acoplamiento referencial extremo, ausencia de pruebas automatizadas y dependencias globales que eliminan cualquier posibilidad de prueba unitaria aislada. Cuando la deuda acumulada supera un umbral crítico, el sistema se convierte en lo que se denomina *big ball of mud* [8]: una masa de código sin estructura discernible donde cualquier modificación introduce efectos colaterales impredecibles.

En un monolito, un fallo en cualquier componente puede comprometer la estabilidad de la aplicación completa. La actualización de una funcionalidad menor requiere el redesplicue del sistema entero, lo que incrementa el riesgo operativo y prolonga los tiempos de indisponibilidad. Además, la ausencia de límites arquitectónicos claros dificulta que nuevos colaboradores comprendan el sistema sin depender del conocimiento tácito de los mantenedores originales.

2.2.2. Arquitecturas Basadas en Microservicios

Para mitigar las deficiencias de los monolitos, la industria evolucionó hacia la arquitectura basada en microservicios. Fowler y Lewis [9] definen los microservicios como un enfoque de desarrollo en el que una aplicación única se construye como una suite de pequeños servicios, donde cada uno ejecuta su propio proceso y se comunica mediante mecanismos ligeros, usualmente APIs HTTP/REST. Estos servicios están contruidos alrededor de capacidades de negocio, son desplegables de manera independiente y pueden utilizar diferentes tecnologías de almacenamiento de datos.

El principio fundamental que sustenta los microservicios es el de *contextos acotados* (bounded contexts), definido por Evans en el *Domain-Driven Design*.

Cada servicio encapsula un modelo de datos y una lógica de negocio coherentes, minimizando las dependencias con los demás. Esta descomposición permite que un nuevo colaborador comprenda y modifique un dominio específico sin necesidad de reconstruir el modelo mental del sistema completo, reduciendo la carga cognitiva que impone una base de código monolítica sobre personal de mantenimiento rotativo.

No obstante, la adopción de microservicios introduce complejidad operativa inherente a los sistemas distribuidos: latencia de red, tolerancia a fallos, monitoreo distribuido, consistencia eventual de datos y orquestación de despliegues [7]. Esta complejidad no es un defecto del enfoque, sino una consecuencia directa del intercambio: se sacrifica simplicidad operativa unitaria a cambio de flexibilidad evolutiva y resiliencia. Sin los mecanismos adecuados de observabilidad y automatización, la complejidad operativa puede superar los beneficios arquitectónicos.

2.2.3. Comparación entre Arquitecturas Monolíticas y Microservicios

La principal diferencia radica en el acoplamiento y la unidad de despliegue. Las arquitecturas monolíticas ofrecen simplicidad operativa en etapas tempranas pero sufren de alto acoplamiento interno. Por el contrario, los microservicios brindan alta cohesión por dominio, escalabilidad independiente por servicio y resiliencia ante fallos aislados. Un servicio caído no compromete la totalidad del ecosistema, siempre que existan mecanismos de tolerancia a fallos en los consumidores.

En el contexto de sistemas académicos con equipos rotativos de mantenimiento, la elección arquitectónica tiene implicaciones prácticas directas. Un monolito obliga a cada nuevo colaborador a comprender la totalidad del sistema antes de poder realizar cualquier modificación, complicando el desarrollo de nuevas características. Un ecosistema de microservicios distribuye la complejidad pero exige que cada servicio sea comprensible de forma aislada y que existan mecanismos automatizados de validación que reduzcan la dependencia del conocimiento individual.

2.3. Tecnologías para el Desarrollo y Operación de Microservicios

La migración a microservicios requiere un ecosistema tecnológico que abarque desde la construcción del código hasta la observación del sistema en producción. A continuación se describen las herramientas y frameworks seleccionados para el desarrollo del presente trabajo, agrupados por función operativa.

2.3.1. Contenedores y Orquestación

Docker es una plataforma de contenedores que empaqueta una aplicación junto con todas sus dependencias —bibliotecas, configuraciones y variables de entorno— en una unidad portable e invariante denominada imagen. Un contenedor ejecuta una instancia de esta imagen de forma aislada sobre el kernel del sistema operativo host, sin requerir una máquina virtual completa. Esta característica reduce drásticamente el consumo de recursos y garantiza que el comportamiento del software sea idéntico en los entornos de desarrollo, prueba y producción.

Docker Compose es una herramienta de orquestación local que permite describir un ecosistema completo de contenedores —microservicios, bases de datos, proxies, herramientas de observabilidad— en un único archivo declarativo. Mediante un comando único se levanta toda la red de contenedores con sus dependencias, volúmenes y configuraciones de red interna. Frente a orquestadores más sofisticados como Kubernetes, Docker Compose ofrece simplicidad operativa para entornos sin requisitos de autoescalado o alta disponibilidad multi-nodo, lo cual resulta adecuado para el contexto de servidores universitarios *on-premise*¹.

2.3.2. Framework y Construcción

La selección del framework de backend determinó gran parte del ecosistema tecnológico del nuevo sistema. Se evaluaron dos alternativas principales.

Node.js con Express.js es el stack del monolito legacy de PortalAsig. Express.js es un framework minimalista para Node.js que permite construir servidores HTTP con poco código boilerplate. Entre sus ventajas se encuentra la curva de aprendizaje reducida para desarrolladores con experiencia en JavaScript, la gran cantidad de paquetes disponibles en npm y el modelo de eventos no bloqueante que maneja eficientemente operaciones de entrada y salida. Sin embargo, el ecosistema Node.js carece de soporte nativo para observabilidad distribuida (Micrometer, Actuator), configuración centralizada (Config Client), seguridad OAuth2/JWT y ORM maduro, lo que obliga a depender de bibliotecas de terceros para funcionalidades críticas. Además, la tipificación débil de JavaScript permite que errores de tipo pasen inadvertidos hasta la ejecución. La alternativa habitual —adoptar TypeScript— introduce una capa adicional de complejidad en la configuración del proyecto, la definición de tipos y la integración con bibliotecas de terceros, incrementando el conocimiento requerido para que un colaborador temporal contribuya de forma segura.

Spring Boot es un framework de desarrollo para la plataforma Java que simplifica la creación de aplicaciones autónomas y de grado empresarial. Proporciona de forma inmediata soporte para seguridad (OAuth2 y JWT), observabilidad (Micrometer y Actuator), comunicación entre servicios (WebClient), configuración centralizada (Config Client) y persistencia (JPA). La tipificación fuerte de Java

¹*On-premise*: modalidad de despliegue donde la infraestructura de software reside en instalaciones propias de la organización, en contraste con modelos de computación en la nube donde los recursos son gestionados por un proveedor externo.

facilita la detección temprana de errores en tiempo de compilación, en contraste con lenguajes dinámicos donde las incompatibilidades de tipos pueden pasar inadvertidas hasta la ejecución. Se seleccionó Spring Boot porque la madurez de su ecosistema para arquitecturas de microservicios, la integración nativa con herramientas de observabilidad y testing, y la abundancia de documentación y desarrolladores disponibles lo convierten en la opción más robusta para un sistema enterprise mantenido por colaboradores académicos temporales.

Spring Cloud es un conjunto de herramientas que extienden Spring Boot para escenarios distribuidos. Entre sus componentes destaca **Spring Cloud Config Server**, que centraliza la gestión de propiedades y secretos en un repositorio Git, permitiendo versionar los cambios de configuración junto al código fuente y simplificando la rotación de credenciales sin requerir redesplicue de los servicios.

Maven es una herramienta de gestión de dependencias y construcción para proyectos Java. Organiza el código mediante un modelo de herencia de proyectos donde un POM padre centraliza las versiones de las dependencias y los plugins. Este mecanismo, denominado *Bill of Materials* (BOM), garantiza consistencia entre los distintos módulos del ecosistema y elimina conflictos de versiones.

Lombok es una biblioteca de anotaciones que reduce el código repetitivo en clases Java. Mediante anotaciones como `@Getter`, `@Setter`, `@Builder` y `@AllArgsConstructor`, Lombok genera automáticamente métodos de acceso, constructores y patrones de creación en tiempo de compilación, eliminando decenas de líneas de código boilerplate por entidad. Esta reducción mejora la legibilidad y disminuye el riesgo de inconsistencias cuando múltiples colaboradores modifican el mismo modelo de datos.

MapStruct es un generador de código que automatiza el mapeo entre objetos de dominio (entidades JPA) y objetos de transferencia (DTOs). En lugar de escribir manualmente conversiones campo por campo, el desarrollador define una interfaz anotada y MapStruct genera la implementación en tiempo de compilación. Esta práctica garantiza que cada cambio en la estructura de una entidad se refleje inmediatamente en su DTO correspondiente, evitando errores de sincronización entre capas.

2.3.3. Seguridad y Comunicación

JSON Web Token (JWT) es un estándar abierto (RFC 7519) que define un formato compacto y autocontenido para transmitir información de autenticación entre partes. Un token JWT contiene tres segmentos codificados en Base64: un encabezado con el algoritmo de firma, un *payload* con las reclamaciones del usuario (identidad, roles, expiración) y una firma criptográfica que garantiza la integridad del mensaje. Los servicios pueden validar un JWT sin necesidad de consultar una base de datos de sesiones en cada petición, lo que habilita la escalabilidad horizontal.

OAuth2 es un protocolo de autorización que permite a una aplicación obtener acceso limitado a los recursos de un usuario sin exponer sus credenciales. El flujo `client_credentials` permite que un microservicio se autentique directamente

con sus propias credenciales para consumir APIs internas, mientras que el flujo de *password* autentica a usuarios finales. Ambos flujos emiten tokens de acceso con tiempo de expiración limitado que los servicios protegidos validan mediante la firma JWT compartida. Para evitar que el usuario deba iniciar sesión repetidamente, el protocolo define un *refresh token*: un token de mayor duración que el cliente puede intercambiar por un nuevo par de tokens de acceso cuando el original expire, manteniendo la sesión activa sin requerir credenciales nuevamente.

Nginx es un servidor web de alto rendimiento que en este contexto opera como proxy inverso: recibe las peticiones del navegador, las enruta hacia el microservicio correspondiente y devuelve la respuesta al cliente. Esta capa intermedia permite servir el frontend estático, distribuir la carga entre réplicas y centralizar la terminación SSL.

Cross-Origin Resource Sharing (CORS) es un mecanismo de seguridad que controla qué dominios externos pueden realizar peticiones HTTP a un servicio. En una arquitectura de microservicios donde el frontend Vue.js se sirve desde un dominio distinto al de los servicios backend, CORS debe configurarse explícitamente en cada microservicio para permitir el intercambio de recursos entre orígenes autorizados.

Cookies y **localStorage** son mecanismos de almacenamiento del lado del cliente. Las cookies permiten que el navegador almacene pares clave-valor asociados a un dominio, con soporte nativo para expiración y atributos de seguridad (**HttpOnly**, **Secure**). **localStorage** ofrece un almacenamiento persistente de mayor capacidad, accesible únicamente mediante JavaScript, pero carece de los mecanismos de expiración y protección contra ataques XSS (cross-site scripting) que proporcionan las cookies. En el contexto de autenticación, las cookies resultan preferibles para almacenar tokens de sesión porque el navegador las envía automáticamente en cada petición y las elimina al expirar, reduciendo la superficie de ataque frente a vulnerabilidades de inyección de scripts.

2.3.4. Persistencia

Java Persistence API (JPA) es una especificación de Java para el mapeo objeto-relacional (ORM). Permite definir las entidades del modelo de datos como clases Java anotadas y delegar la generación de consultas SQL al framework. **Hibernate** es la implementación de referencia de JPA en el ecosistema Spring Boot. Esta combinación abstrae las diferencias de dialecto entre motores de base de datos y reduce la cantidad de código SQL explícito que el desarrollador debe escribir.

Flyway es una herramienta de migración de bases de datos que aplica scripts SQL versionados durante el arranque de cada servicio. Los scripts se organizan en el directorio `db/migration/` con un prefijo de versión obligatorio (`V1_`, `V2_`), lo que garantiza que el esquema de la base de datos esté siempre alineado con la versión del código desplegado. Este enfoque elimina las migraciones manuales y permite reproducir el estado de la base de datos en cualquier entorno a partir del repositorio de código fuente.

MySQL es un motor de base de datos relacional maduro, con integración nativa en Spring Boot a través del conector oficial `mysql-connector-j`, herramientas de administración accesibles y documentación extensa. En una arquitectura de microservicios se adopta el principio de una instancia aislada por servicio, eliminando el acoplamiento referencial del modelo monolítico y permitiendo que cada dominio evolucione su esquema de forma independiente.

PostgreSQL es un motor de base de datos relacional con soporte avanzado para tipos de datos complejos, consultas anidadas y extensibilidad. En el contexto de PortalAsig, PostgreSQL fue el motor utilizado por el backend monolítico original, que centralizaba todas las entidades del sistema en un único schema sin aislamiento por dominio. La migración a MySQL responde a la necesidad de alinear el stack tecnológico con el ecosistema Spring Boot y a la madurez de las herramientas de administración disponibles para entornos universitarios.

2.3.5. Observabilidad Distribuida

La observabilidad de un sistema distribuido se fundamenta en tres pilares de telemetría: métricas, logs y trazabilidad. Sridharan (2018) [10] distingue entre *monitoreo* —el acto de observar el estado general y predecible de un sistema— y la *observabilidad* —la cualidad que permite inferir estados internos basándose únicamente en el conocimiento de sus salidas externas.

Spring Boot Actuator expone endpoints de administración y diagnóstico sobre cada microservicio, incluyendo `/actuator/health` para verificar el estado de salud y `/actuator/prometheus` para exportar métricas en formato compatible con Prometheus. **Micrometer** es la fachada de instrumentación que Actuator utiliza internamente para recolectar métricas de JVM, uso de memoria, tiempos de respuesta y contadores de peticiones HTTP.

Prometheus es un sistema de monitoreo y alerta que recolecta métricas mediante un modelo de extracción periódica (*scraping*) sobre los endpoints expuestos por los microservicios. Almacena las series temporales en una base de datos propia y permite consultarlas mediante un lenguaje de expresiones (PromQL).

Loki es un sistema de agregación de logs diseñado específicamente para entornos distribuidos. A diferencia de soluciones como Elasticsearch, que indexan el contenido completo de cada registro, Loki indexa únicamente las etiquetas (*labels*) asociadas a los logs, logrando un consumo de recursos significativamente menor. Los microservicios envían sus registros a Loki mediante un *appender*² configurado en el framework de logging.

Grafana es una plataforma de visualización que unifica métricas de Prometheus y logs de Loki en dashboards interactivos. Permite correlacionar anomalías numéricas —como un pico de latencia— con el contexto textual de los logs en un único punto de consulta, acelerando el diagnóstico de incidentes.

²*Appender*: componente de un framework de *logging* que define el destino de los registros (consola, archivo, servidor remoto, etc.).

2.3.6. Calidad de Software y Testing

La migración a microservicios no garantiza por sí sola la mantenibilidad del sistema. Es necesario incorporar prácticas sistemáticas de aseguramiento de calidad que detecten problemas antes de que lleguen a producción.

Checkstyle es una herramienta de análisis estático de código que verifica el cumplimiento de una guía de estilo definida en un archivo de configuración XML. Se integra en la fase de compilación de Maven de forma que cualquier violación —longitud de línea excedida, convenciones de nombres incorrectas, ausencia de Javadoc— interrumpe el *build*. Esta decisión prioriza la legibilidad y uniformidad del código fuente, aspectos críticos cuando múltiples colaboradores rotativos deben leer y modificar el mismo código.

JaCoCo es un plugin de cobertura de código para Java que instrumenta el *bytecode* durante la ejecución de las pruebas y genera un reporte detallado del porcentaje de líneas y ramas ejecutadas. El reporte se integra con el pipeline de integración continua para bloquear la fusión de código que descienda por debajo del umbral establecido.

Testcontainers es una biblioteca que levanta contenedores Docker efímeros durante la fase de pruebas de Maven. Permite ejecutar tests de integración sobre instancias reales de MySQL, aplicar las migraciones Flyway correspondientes y destruir el contenedor al finalizar. Este enfoque garantiza que las pruebas validen comportamientos que dependen del dialecto SQL real, como restricciones de clave foránea y transacciones concurrentes, en contraste con bases de datos en memoria como H2.

Se evaluaron tres alternativas para la automatización de pruebas end-to-end.

Selenium WebDriver constituye el estándar más consolidado de la industria. Opera mediante un protocolo de comunicación entre un driver específico del navegador y el código de prueba, lo que permite ejecutar tests en múltiples navegadores y plataformas. Sin embargo, esta arquitectura de intermediación introduce latencia adicional y condiciones de sincronización entre el driver y el navegador que, en sistemas de interfaz compleja, producen inestabilidad en las pruebas.

Playwright es una herramienta más reciente que soporta múltiples navegadores (Chromium, WebKit, Firefox) mediante una arquitectura de eventos que permite ejecución paralela rápida. Su API es robusta y su soporte multi-navegador supera al de Cypress. No obstante, su ecosistema es relativamente nuevo y su curva de aprendizaje es ligeramente mayor para equipos sin experiencia previa en automatización de interfaz.

ESLint es un analizador estático de código JavaScript que detecta patrones problemáticos, violaciones de convenciones de estilo y potenciales errores de lógica antes de la ejecución. Se integra en el ciclo de desarrollo del frontend mediante un script de **npm** que verifica el código antes de compilar la aplicación, garantizando que solo código que cumple con el estándar definido llegue al repositorio.

Prettier es un formateador de código que aplica un estilo consistente a través del proyecto. A diferencia de ESLint, que detecta problemas semánticos,

Prettier se encarga exclusivamente de la presentación: indentación, saltos de línea, comillas y espaciado. La combinación de ESLint y Prettier establece un estándar de calidad reproducible para el código JavaScript del frontend, reduciendo las discrepancias de estilo entre colaboradores y eliminando debates sobre formato en las revisiones de código.

Cypress se ejecuta dentro del mismo *runtime* del navegador, eliminando por completo la capa de intermediación del WebDriver. Esta característica produce tiempos de respuesta más predecibles y reduce drásticamente la tasa de falsos negativos producidos por condiciones de carrera en la interfaz. Además, Cypress incluye mecanismos de espera automática integrados que eliminan la necesidad de declarar tiempos de espera explícitos en el código de prueba. Se seleccionó Cypress porque el balance entre velocidad, estabilidad y facilidad de uso resulta óptimo para un proyecto académico donde el tiempo de desarrollo de pruebas es un recurso limitado y donde el ecosistema Vue.js cuenta con integración nativa con esta herramienta.

Para la integración continua se evaluaron dos alternativas.

Jenkins es una plataforma de integración continua autoalojada que ofrece una amplia gama de plugins y un alto grado de configurabilidad. Su principal ventaja es la capacidad de adaptarse a flujos de trabajo complejos y entornos empresariales. Sin embargo, requiere un servidor propio, configuración manual de plugins, actualizaciones periódicas y mantenimiento de la infraestructura. En un contexto sin personal de operaciones dedicado, esta complejidad operativa representa un pasivo técnico que el siguiente colaborador deberá asumir.

GitHub Actions es un servicio de integración continua integrado en GitHub que ejecuta pipelines definidos mediante archivos YAML en el mismo repositorio del código. El flujo de trabajo se dispara automáticamente en cada *pull request* y en cada cambio sobre la rama principal, realizando en secuencia: compilación, análisis de estilo, ejecución de pruebas y generación del reporte de cobertura. Se seleccionó GitHub Actions sobre Jenkins porque la ejecución corre sobre los servidores de GitHub sin costo adicional para repositorios públicos, eliminando la necesidad de infraestructura propia y reduciendo la carga operativa sobre el equipo de mantenimiento. Las ramas principales de cada repositorio se protegen mediante reglas de *branch protection* que exigen el paso exitoso de la pipeline antes de permitir la fusión del código.

k6 es una herramienta de código abierto para pruebas de carga que permite simular tráfico concurrente sobre APIs HTTP mediante scripts escritos en JavaScript. A diferencia de herramientas de pruebas funcionales que verifican la corrección del comportamiento, k6 mide el rendimiento del sistema bajo condiciones de estrés: latencia percentil, throughput y tasa de errores ante un número creciente de usuarios virtuales. Se emplea para comparar el comportamiento del monolito legacy y el ecosistema de microservicios bajo cargas sintéticas equivalentes, generando métricas reproducibles que alimentan el análisis de diseño experimental del Capítulo 4.

2.3.7. Documentación y Frontend

OpenAPI es una especificación estándar para describir APIs REST de forma declarativa. **SpringDoc OpenAPI** es una biblioteca que genera automáticamente una interfaz interactiva de documentación (Swagger UI) a partir de las anotaciones presentes en los controladores REST. Esta documentación viva se actualiza con cada cambio en el código, eliminando la necesidad de mantener manuales de API separados.

Vue.js es un framework progresivo para la construcción de interfaces de usuario. A diferencia de frameworks monolíticos, Vue permite adoptar sus características de forma incremental: un desarrollador puede integrar un componente Vue en una página HTML existente o construir una aplicación de página única (SPA) completa. Su modelo de reactividad y su ecosistema de herramientas (Vuex para gestión de estado, Vue Router para navegación) facilitan la construcción de interfaces tolerantes a fallos que absorben errores de red sin bloquear la experiencia del usuario.

Axios es un cliente HTTP basado en promesas para JavaScript. En el frontend de PortalAsig, Axios gestiona la comunicación entre la interfaz de usuario y los microservicios backend, permitiendo configurar instancias independientes con URL base, tiempos de espera y cabeceras personalizadas por servicio. Esta capacidad resulta esencial en una arquitectura distribuida donde el frontend debe comunicarse con múltiples endpoints (UAA, Site, PAIMA) sin depender de un único punto de entrada.

Bootstrap-Vue es una biblioteca de componentes de interfaz de usuario que integra el framework CSS Bootstrap con Vue.js. Proporciona componentes accesibles y responsivos (tablas, formularios, modales, navegación) que aceleran el desarrollo del frontend sin requerir la escritura manual de estilos CSS complejos. Su adopción garantiza consistencia visual en toda la aplicación y reduce el tiempo necesario para implementar nuevas vistas.

2.3.8. Modelos de Lenguaje de Gran Escala y Locales

Los **modelos de lenguaje de gran escala** (LLMs, por sus siglas en inglés) son redes neuronales entrenadas sobre vastos corpus de texto que pueden generar, resumir, traducir y razonar sobre lenguaje natural. En el contexto de operaciones de infraestructura, los LLMs actúan como asistentes que interpretan telemetría operativa —métricas, logs, topología de servicios— y sintetizan diagnósticos legibles por humanos. Reciben instrucciones de texto denominadas *prompts*, que estructuran el contexto técnico en un formato que el modelo puede procesar.

Los proveedores de modelos de lenguaje pueden clasificarse en dos categorías según su modalidad de ejecución.

APIs remotas comerciales. Servicios como OpenAI (GPT-4, GPT-3.5), Google (Gemini) o Moonshot AI (Kimi) ofrecen acceso a modelos de gran capacidad mediante interfaces de programación en la nube. Estos modelos han sido entrenados sobre vastos corpus de texto y exhiben un alto rendimiento en tareas de razonamiento, síntesis y generación de código. Su principal ventaja es

la calidad de respuesta sin requerir infraestructura local de procesamiento. Su principal desventaja es la dependencia de conectividad externa, los costos por token consumido y la transmisión de datos operativos potencialmente sensibles a servidores de terceros.

Modelos locales. Ollama es una plataforma de código abierto que permite ejecutar modelos de lenguaje de forma local sobre hardware convencional, sin depender de APIs de terceros. Modelos como Llama, Gemma u otros pueden ejecutarse en estaciones de trabajo locales con recursos limitados, garantizando soberanía tecnológica y eliminando costos por uso de APIs remotas. Esta capacidad resulta especialmente relevante en entornos institucionales donde la conectividad externa es limitada o donde los datos operativos no pueden transmitirse a servicios en la nube por razones de seguridad. La contraparte es que los modelos locales suelen tener menor capacidad de razonamiento que sus equivalentes comerciales de mayor escala.

En el contexto de este trabajo, se adoptó una arquitectura de proveedores configurables que permite alternar entre APIs remotas (para máxima calidad de diagnóstico) y modelos locales (para soberanía tecnológica), seleccionable mediante el Config Server sin modificación del código.

2.4. Mantenimiento de Software

2.4.1. Tipos de Mantenimiento

El estándar internacional ISO/IEC/IEEE 14764 [11] establece que el mantenimiento del software ocurre una vez que el sistema ha sido puesto en producción y lo categoriza en cuatro vertientes:

- **Correctivo:** Modificaciones reactivas para corregir fallos o *bugs* descubiertos durante la operación.
- **Adaptativo:** Modificaciones para asegurar que el software siga siendo funcional en un entorno tecnológico cambiante (migraciones de base de datos, actualizaciones de sistema operativo, cambios en dependencias).
- **Perfectivo:** Mejoras en el rendimiento o implementación de nuevas funcionalidades operativas sin alterar la lógica de negocio subyacente.
- **Preventivo:** Refactorización o mitigación técnica para prevenir fallos latentes o corregir la deuda técnica antes de que cause incidencias.

2.4.2. Retos en Sistemas Distribuidos

Realizar tareas de mantenimiento en arquitecturas basadas en microservicios introduce retos significativos. La falla no suele concentrarse en una sola línea de código, sino que puede deberse a interrupciones de red, fallos en contenedores terceros o latencias asíncronas [7]. Identificar la raíz del problema (Root Cause

Analysis, RCA) consume una porción abrumadora del tiempo del desarrollador, lo cual incrementa el MTTR (Mean Time to Recovery).

En un ecosistema distribuido, la observabilidad no es un lujo sino un requisito operativo. Sin métricas por servicio, sin agregación centralizada de logs y sin trazabilidad distribuida, el diagnóstico se convierte en un proceso de ensayo y error que depende del conocimiento tácito del mantenedor. Esta dependencia es precisamente lo que la transición hacia la Web 3.0 busca eliminar, reemplazando el mantenimiento reactivo por ciclos de auto-gestión asistidos.

2.4.3. Prácticas Modernas de Aseguramiento de Calidad

Las técnicas de mantenimiento moderno trascienden la corrección de errores para abarcar la prevención sistemática de defectos y la reducción de la deuda técnica. La **integración continua** (CI) automatiza la compilación, el análisis estático y la ejecución de pruebas cada vez que un desarrollador propone un cambio, impidiendo que código que no cumple con los criterios de calidad llegue a la línea base del proyecto. En el contexto de este trabajo, el alcance de la integración continua se limita a la fase de validación automatizada; el despliegue continuo en entornos de preproducción o producción, conocido como entrega continua (CD), queda fuera del alcance dado que la infraestructura de despliegue automático requiere una arquitectura de orquestación que el presente trabajo no implementa.

El **análisis de cobertura de código** mide el porcentaje de líneas y ramas condicionales ejecutadas por la suite de pruebas. Un umbral mínimo de cobertura, validado automáticamente en cada *pull request*, impide que código no probado llegue a la línea base del proyecto. Las reglas de **protección de ramas** bloquean el envío directo de código a la rama principal, exigiendo que todo cambio pase por revisión humana y validación automatizada antes de su fusión.

Estas prácticas, en conjunto, transforman el mantenimiento de un proceso artesanal y dependiente del conocimiento tácito en un proceso sistemático, medible y reproducible. Constituyen la base sobre la cual un sistema puede evolucionar hacia la autonomía operativa, ya que la auto-adaptación requiere que el código alterado mantenga su validez estructural.

2.5. Computación Autonómica

2.5.1. Concepto y Propiedades Self-CHOP

Inspirada en el funcionamiento del sistema nervioso autónomo del cuerpo humano, la Computación Autonómica fue propuesta por IBM en 2001 para lidiar con la creciente complejidad de la gestión de sistemas informáticos [12]. El concepto define sistemas informáticos capaces de autogestionarse en base a políticas de alto nivel establecidas por administradores humanos, ocultando la complejidad subyacente de las operaciones técnicas.

Un sistema autónomo se rige por cuatro propiedades fundamentales, conocidas como Self-CHOP [13]:

- **Auto-configuración (Self-configuration):** Capacidad de instalar y adaptar componentes de software automáticamente sin intervención externa.
- **Auto-recuperación (Self-healing):** Habilidad para detectar, diagnosticar y reparar problemas operativos, interrupciones y fallos en el software o hardware.
- **Auto-optimización (Self-optimization):** Monitorización proactiva y ajuste continuo de los recursos para garantizar un rendimiento óptimo.
- **Auto-protección (Self-protection):** Capacidad de anticipar, detectar e identificar ataques maliciosos o fallos en cascada, tomando acciones para salvaguardar el sistema.

2.6. Modelo MAPE-K

2.6.1. Componentes del Ciclo

El modelo MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) es el marco arquitectónico de referencia introducido por IBM en 2003 para construir sistemas autónomos [12]. Divide el proceso de auto-gestión en cuatro fases apoyadas sobre una base de conocimiento compartida:

- **Monitor (Monitoreo):** Recopila información de topología, métricas y logs de los recursos gestionados.
- **Analyze (Análisis):** Procesa la información para entender el estado actual del sistema y determinar si se ha producido una anomalía.
- **Plan (Planificación):** Estructura las acciones necesarias para alcanzar los objetivos o solucionar la falla detectada.
- **Execute (Ejecución):** Aplica el plan a través de actuadores o comandos sobre el sistema objetivo.
- **Knowledge (Conocimiento):** Base de datos o repositorio histórico que nutre y es nutrido por las otras cuatro fases [14].

2.6.2. De MAPE-K a la Integración de Inteligencia Artificial

La implementación estricta de un único ciclo MAPE-K mediante reglas estáticas resulta efectiva en entornos controlados con patrones de fallo predecibles. Sin embargo, la transición hacia microservicios introdujo nuevos niveles de

complejidad: múltiples puntos de fallo independientes, interacciones no lineales entre servicios y volúmenes de telemetría que superan la capacidad de análisis manual.

En este contexto, la literatura reciente ha propuesto enfoques híbridos donde la inteligencia artificial asume la función de razonador en las fases de Análisis y Planificación del ciclo MAPE-K. Esposito et al. [15] proponen un framework que integra MAPE-K con *agentic AI* para la detección y remediación autónoma de anomalías en sistemas de microservicios. Su trabajo demuestra que la combinación del ciclo MAPE-K con capacidades de inteligencia artificial generativa permite mantener la robustez y seguridad de ecosistemas distribuidos, reduciendo el tiempo de inactividad y monitoreando atributos de calidad como rendimiento, resiliencia y gestión de anomalías. En este enfoque, las fases de Análisis y Planificación se delegan a un modelo de lenguaje que interpreta el contexto completo de un incidente —métricas, logs, topología de servicios— y propone un plan de acción adaptado, en lugar de depender de umbrales fijos configurados por un operador humano.

Sanwou, Quinton y Temple [16] extienden esta dirección hacia arquitecturas completamente distribuidas con su marco AWARE (*Assess, Weigh, Act, Reflect, Enrich*), que emplea agentes de IA autónomos capaces de colaboración entre nodos y aprendizaje continuo. Si bien AWARE representa una evolución prometedora, su implementación requiere una infraestructura multi-agente que excede los recursos disponibles en el contexto de este trabajo.

El servicio PAIMA (Sección 3.2) se configura como una **implementación práctica del framework propuesto en [15]**: mantiene la arquitectura centralizada del ciclo MAPE-K —un servicio con scheduling periódico que ejecuta Monitor, Analyze, Plan y Execute en secuencia— pero sustituye las reglas estáticas de las fases de Análisis y Planificación por un modelo de lenguaje de gran escala. Esta decisión de diseño prioriza la operabilidad en un entorno sin equipo de operaciones dedicado: un servicio centralizado es más simple de desplegar, depurar y mantener que una red distribuida de agentes autónomos, al tiempo que incorpora la capacidad de razonamiento inteligente que caracteriza a la evolución del MAPE-K hacia la inteligencia artificial.

La fase Knowledge adquiere un rol central en esta implementación, ya que el historial de incidentes resueltos y las decisiones humanas aprobadas constituyen material de retroalimentación que permite refinar los *prompts* enviados al modelo de IA y mejorar progresivamente la calidad de los diagnósticos.

2.7. Sistemas Auto-adaptativos

Un sistema auto-adaptativo tiene la capacidad de modificar su comportamiento y estructura en tiempo de ejecución en respuesta a cambios en su entorno u operaciones. Weyns (2019) establece que un sistema verdaderamente auto-adaptativo requiere retroalimentación constante y mecanismos de toma de decisiones no deterministas que reduzcan la carga cognitiva de los operadores humanos [17].

Mientras que la computación autónoma abarca la visión completa de sistemas auto-manejables, la auto-adaptación es el mecanismo de ingeniería de software práctico para implementarla. La auto-adaptación requiere que el código alterado mantenga su validez estructural, lo cual hace que las prácticas de aseguramiento de calidad —cobertura de pruebas, análisis estático, integración continua— sean condiciones previas indispensables para cualquier forma de autonomía operativa.

2.8. Inteligencia Artificial aplicada al Mantenimiento de Software

El uso de algoritmos de aprendizaje automático e inteligencia artificial permite establecer líneas base del comportamiento normal de un sistema: tasa de errores habitual, latencia esperada, picos estacionales de tráfico. Cuando los patrones reales se desvían de estas líneas base predictivas, la IA puede alertar al equipo antes de que ocurra una interrupción total del servicio [18].

El advenimiento de los modelos de lenguaje de gran escala ha revolucionado la gestión de operaciones de TI. Al abstraer métricas matemáticas, archivos de log crudos y topología de servicios en representaciones de texto, los LLMs son capaces de sintetizar errores complejos en diagnósticos legibles por humanos. Actúan como asistentes para los ingenieros de operaciones, sugiriendo configuraciones óptimas para recuperar o escalar la infraestructura.

2.9. AIOps y Automatización de Operaciones

El término AIOps (Artificial Intelligence for IT Operations) fue acuñado por Gartner en 2016 para describir la aplicación de Big Data, aprendizaje automático y otras tecnologías avanzadas a los flujos de operaciones de TI. AIOps mejora el monitoreo tradicional al correlacionar eventos ruidosos, encontrar la causa raíz con mayor velocidad y proporcionar remediación predictiva.

En ecosistemas orquestados por contenedores, AIOps puede predecir cuándo un servicio fallará analizando un aumento anormal de la memoria o la latencia. La integración de AIOps en modelos autónomos como MAPE-K representa el estado del arte en ingeniería de resiliencia, permitiendo que los sistemas no solo reaccionen ante fallos, sino que anticipen condiciones de riesgo antes de que se materialicen en interrupciones.

2.10. Integración de IA en Arquitecturas de Microservicios

La principal barrera para incorporar inteligencia artificial en operaciones es la fragmentación de los datos. En un entorno de microservicios, los registros están distribuidos en múltiples nodos. Para que un modelo de lenguaje o un modelo

estadístico funcione, se requiere la implementación de patrones de agregación mediante herramientas como Loki y Prometheus, garantizando un punto de entrada centralizado de contexto [19].

Al abstraer las métricas y los logs como contexto para modelos generativos, los administradores obtienen diagnósticos en lenguaje natural. Esto democratiza la administración de servidores complejos y acelera drásticamente los tiempos de recuperación. Permite establecer un canal de auto-mantenimiento donde el sistema puede inferir su propio nivel de riesgo, recomendando a los equipos humanos ajustar configuraciones antes de que una anomalía se convierta en una caída masiva.

En el contexto de la Web 3.0, esta integración no es una funcionalidad adicional sino una capacidad arquitectónica fundamental. Un sistema distribuido que no puede observarse a sí mismo, que no puede agregar sus propios registros y que no puede delegar el análisis a un modelo inteligente, permanece anclado en el paradigma de la Web 2.0: interactivo, pero no adaptativo. La transición hacia la autonomía requiere, como condición necesaria, una arquitectura que haga posible la observabilidad distribuida y la asistencia inteligente.

Capítulo 3

Marco Aplicativo

La arquitectura monolítica de PortalAsig acumula seis años de modificaciones incrementales sin un proceso sistemático de mantenimiento, generando un grado de acoplamiento que dificulta cualquier intervención por parte de colaboradores académicos temporales. Para superar esta situación, se diseñó e implementó un nuevo backend basado en microservicios que descompone el sistema en dominios independientes, cada uno con su propia base de datos, su ciclo de despliegue autónomo y mecanismos de validación automatizada que reducen la dependencia del conocimiento tácito. Sobre este ecosistema distribuido se construyó el servicio PAIMA, que asume la función de operador autónomo mediante un ciclo de control MAPE-K asistido por inteligencia artificial, monitoreando métricas y logs, diagnosticando anomalías y proponiendo acciones de remediación que mitigan la carga cognitiva sobre el mantenedor humano.

3.1. Migración del Backend a una Arquitectura de Microservicios

3.1.1. Análisis del Sistema Monolítico Legacy

El backend de PortalAsig en producción se ejecuta sobre la plataforma Node.js, empleando el framework Express.js como servidor de aplicaciones, el ORM Bookshelf.js sobre el constructor de consultas Knex.js, y una base de datos PostgreSQL como único almacén de datos. Desde su puesta en operación en el año 2016, esta arquitectura monolítica atiende todas las funcionalidades del sistema —autenticación de usuarios, gestión de asignaturas, creación de sitios académicos, administración de evaluaciones, foros, entregas de archivos, envío de correos y encuestas— dentro de un único proceso desplegable.

Aunque este enfoque simplificó el desarrollo inicial y la puesta en marcha, la evolución del sistema durante múltiples semestres académicos sin un proceso sistemático de mantenimiento ha generado un acoplamiento progresivo que afecta la capacidad de modificación del software. En particular, se identifican las

siguientes deficiencias estructurales que justifican la migración a una arquitectura modular.

Acoplamiento total en un único proceso. Todas las responsabilidades del backend se ejecutan dentro de un solo proceso Node.js, lo que implica que un fallo en cualquier módulo —por ejemplo, el procesamiento de archivos adjuntos— puede comprometer la estabilidad de toda la aplicación. No existe mecanismo de aislamiento que permita que un dominio de negocio falle sin afectar a los demás.

Controladores de dimensiones críticas. El archivo `site.js`, ubicado en el directorio `api/controllers/`, concentra la lógica de operaciones de sitios académicos, objetivos, bibliografías, contenidos temáticos, grupo docente, estudiantes, horarios, evaluaciones, calificaciones, foros, comentarios, entregas, archivos, vídeos, enlaces, planificaciones, noticias y encuestas. Este controlador alcanza un total de 3.629 líneas de código, convirtiéndolo en un componente de difícil comprensión y alto riesgo de introducir efectos colaterales ante cualquier modificación.

Modelo de datos con acoplamiento referencial extremo. La entidad `Site`, definida en el archivo `api/models/site.js`, declara explícitamente dieciocho relaciones con otras entidades del sistema. La eliminación de un sitio requiere la ejecución manual de dieciocho operaciones de borrado en cascada dentro de un hook del modelo, implementado mediante promesas anidadas. Si alguna de estas operaciones falla, el sistema queda en un estado de inconsistencia parcial, sin mecanismo de compensación automática.

Autenticación propietaria no estandarizada. El sistema no emplea tokens JWT ni protocolos OAuth2. En su lugar, genera cadenas aleatorias que se almacenan en una tabla `Session` de PostgreSQL, y cada petición al backend requiere una consulta a esta tabla para validar la sesión activa. Este mecanismo impide la escalabilidad horizontal y complica la integración con estándares de seguridad modernos.

Dependencias gestionadas de forma global. El archivo principal `app.js` instancia todos los componentes del sistema —conexión a base de datos, modelos ORM, servicio de correo, utilidades de cifrado— y los inyecta en un contenedor global `app.locals`¹, desde donde los controladores acceden directamente. Esta práctica elimina cualquier posibilidad de prueba unitaria aislada y dificulta la identificación de las dependencias reales de cada módulo.

Ausencia de prácticas de aseguramiento de calidad. El repositorio no cuenta con un directorio de pruebas automatizadas, ni pipelines de integración continua, ni herramientas de análisis estático de código. El despliegue en producción se realiza de forma manual directamente sobre el servidor del Centro de Computación de la Facultad de Ciencias, sin mecanismos que validen el comportamiento del sistema después de cada cambio.

Observabilidad básica. El sistema no expone endpoints de métricas, no cuenta con agregación centralizada de logs ni trazabilidad distribuida. La única forma de detectar un problema es mediante la observación directa del compor-

¹`app.locals`: objeto global proporcionado por Express.js para compartir datos entre *middlewares* y controladores durante el ciclo de vida de la aplicación.

tamiento de la aplicación por parte del usuario final o del mantenedor, lo que convierte el diagnóstico en un proceso enteramente reactivo y dependiente del conocimiento tácito del administrador.

En conjunto, estas deficiencias estructurales han generado un sistema cuya capacidad de adaptación es extremadamente limitada. Cualquier modificación requiere comprender la totalidad del código, con un riesgo elevado de introducir fallos imprevistos. Esta situación, sumada a la ausencia de un equipo de desarrollo de tiempo completo y a la dependencia de colaboradores académicos temporales, exige un rediseño arquitectónico que descomponga el sistema en dominios independientes, con límites claros y mecanismos de validación automatizada.



Figura 3.1: Diagrama entidad-relación simplificado del modelo de datos monolítico, mostrando la tabla **site** como núcleo con sus 15+ relaciones hacia entidades dependientes.

3.1.2. Diseño de la Arquitectura de Microservicios

Para superar las limitaciones del backend monolítico, se diseñó una arquitectura basada en microservicios que descompone el sistema en dominios independientes, cada uno con responsabilidades bien definidas y desplegable de forma autónoma. Este rediseño se fundamenta en el principio de *contextos acotados*, definido en el Marco Teórico, que establece que cada servicio debe encapsular un modelo de datos y una lógica de negocio coherentes, minimizando las dependencias con los demás.

Se identificaron tres contextos fundamentales que agrupan las funcionalidades del sistema:

- **Gestión de Identidad y Acceso (UAA):** Responsable de la autenticación, autorización y emisión de tokens JWT. Centraliza la información de usuarios, roles y clientes OAuth2, permitiendo que cualquier otro servicio valide identidades sin replicar esta lógica.
- **Gestión de Sitios Académicos (Site):** Contiene la lógica de negocio central del portal: asignaturas, semestres, sitios de curso, participantes, secciones, horarios, evaluaciones, contenidos temáticos y referencias bibliográficas.
- **Servicio de Notificaciones (Notify):** Encargado del procesamiento asíncrono y despacho de correos electrónicos, desacoplando el envío de notificaciones de las operaciones síncronas de los demás servicios.

El ecosistema incluye además el servicio **PAIMA**, que opera como un nodo autónomo encargado del ciclo de autogestión del mantenimiento y se detalla en la Sección 3.2.

Principio de base de datos por servicio. Cada microservicio posee su propia instancia de base de datos MySQL, aislada físicamente de las demás. Esta decisión garantiza que un cambio en el modelo de datos de un dominio no afecte a los otros, elimina el riesgo de bloqueos entre servicios y permite que cada estudiante o colaborador que forme parte del desarrollo y mantenimiento del portal pueda evolucionar el modelo de datos de un servicio sin afectar dominios no relacionados. Las bases de datos se identifican como `mysql-uaa`, `mysql-site`, `mysql-notify` y `mysql-paima`, cada una expuesta en un puerto distinto del host para facilitar el diagnóstico local.

Configuración centralizada. Para evitar la dispersión de propiedades y secretos en múltiples archivos, se implementó un servidor de configuración basado en Spring Cloud Config Server. Este componente resuelve las propiedades de cada microservicio desde un repositorio Git local, permitiendo versionar los cambios de configuración junto al código fuente y simplificando la rotación de credenciales. Los servicios obtienen su configuración al iniciarse mediante una única variable de entorno que apunta al *Config Server*, reduciendo la complejidad del despliegue.

Punto de entrada y comunicación. El frontend de la aplicación, desarrollado en Vue.js, se sirve a través de **Nginx**, que actúa como proxy inverso

enrutando las peticiones del navegador hacia los microservicios correspondientes. La comunicación interna entre los servicios se realiza directamente a través de la red virtual de Docker Compose, utilizando el nombre del contenedor como host (por ejemplo, `http://ms-uaa:5860`). Si bien un API Gateway constituye una práctica recomendada en arquitecturas de microservicios para centralizar autenticación, enrutamiento y *limitación de tasa de peticiones*, su implementación quedó fuera del alcance de este trabajo, priorizándose la funcionalidad central del ecosistema y del servicio de autogestión del mantenimiento.

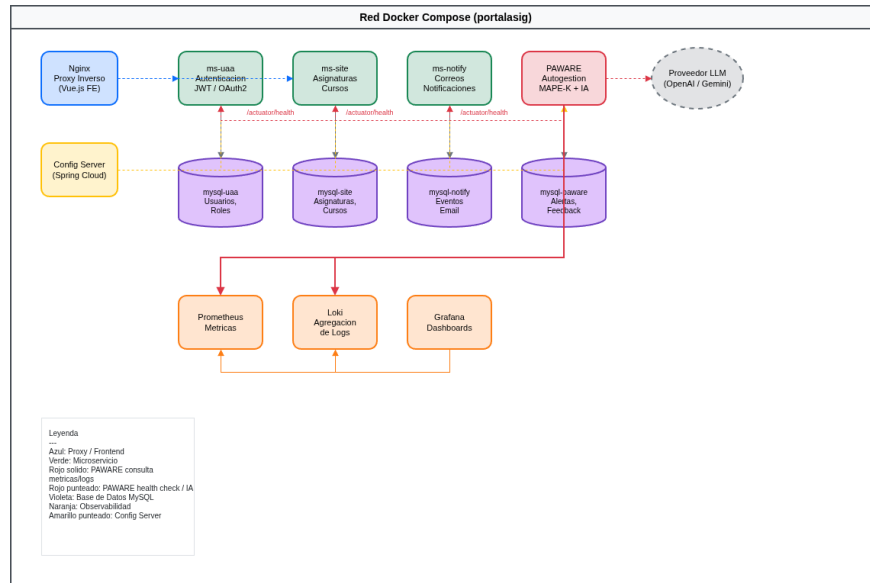


Figura 3.2: Arquitectura del ecosistema de microservicios de PortalAsig, mostrando el proxy Nginx, el Config Server, los cuatro microservicios (UAA, Site, Notify, PAIWARE), sus bases de datos MySQL aisladas, el proveedor LLM externo, el stack de observabilidad (Prometheus, Loki, Grafana) y las conexiones de monitoreo de PAIWARE hacia los endpoints `/actuator/health` de cada microservicio, todo desplegado sobre la red virtual de Docker Compose.

3.1.3. Stack Tecnológico del Backend

La selección del stack tecnológico respondió al criterio de maximizar la madurez del ecosistema, la disponibilidad de documentación y la simplicidad operativa, de forma que un colaborador académico temporal pueda comprenderlo y modificarlo sin depender de conocimiento tácito previo. Se priorizaron herramientas consolidadas, ampliamente documentadas y alineadas con los estándares de la industria, evitando la adopción de tecnologías experimentales que pudieran convertirse en pasivos técnicos.

Lenguaje y framework. Se seleccionaron **Java 17** y **Spring Boot 3**

como base para todos los microservicios. Estas versiones correspondían a las *Long Term Support* (LTS)² más recientes disponibles al inicio del desarrollo de este trabajo, lo que garantizó soporte extendido, estabilidad y acceso a las mejoras de rendimiento y seguridad introducidas en ambas plataformas. Aunque el monolito original utiliza Node.js y Express.js, el ecosistema Spring Boot ofrece de forma nativa (*out-of-the-box*)³ soporte para seguridad (OAuth2 y JWT), observabilidad (Micrometer, Actuator), comunicación entre servicios (WebClient) y configuración centralizada (Config Client), lo que elimina la dependencia de bibliotecas de terceros para funcionalidades críticas del ecosistema [19]. La tipificación fuerte de Java facilita la detección temprana de errores en tiempo de compilación —como incompatibilidades de tipos en parámetros de métodos o acceso a propiedades inexistentes— que en JavaScript suelen pasar inadvertidos hasta la ejecución en producción.

Gestión de dependencias y construcción. Todos los proyectos se gestionan con **Maven**, empleando un modelo de herencia de proyectos que garantiza consistencia entre los distintos nodos del ecosistema. Se diseñó el módulo **portalsig-commons** como biblioteca compartida, estructurado en tres artefactos:

- **core-boot**: POM⁴ padre con el *Bill of Materials* (BOM)⁵ de Spring Boot y Spring Cloud. Define las versiones de todas las dependencias del ecosistema, eliminando conflictos de versiones entre microservicios.
- **core-lib**: Código reutilizable incluyendo configuración de seguridad JWT, manejo centralizado de excepciones, serialización JSON, CORS⁶ y clientes HTTP con soporte para **client_credentials**⁷ y **bearer** tokens⁸.
- **core-backend**: POM padre específico para microservicios backend, que hereda de **core-boot**, incluye **core-lib** y añade dependencias de persistencia (JPA, Flyway, conector MySQL), observabilidad (Micrometer Prometheus, *appender*⁹ Loki) y utilidades de compilación (Lombok, MapStruct).

Esta estructura evita la duplicación de configuración y garantiza que cualquier

²LTS (*Long Term Support*): versión de software que recibe actualizaciones de seguridad y correcciones durante un período extendido, típicamente varios años.

³*Out-of-the-box*: expresión que indica que una funcionalidad está disponible de forma inmediata, sin requerir configuración adicional.

⁴POM (*Project Object Model*): archivo descriptor XML en Maven que define la configuración, dependencias y ciclo de vida de un proyecto Java.

⁵BOM (*Bill of Materials*): en Maven, mecanismo que centraliza y alinea las versiones de un conjunto de dependencias relacionadas.

⁶CORS (*Cross-Origin Resource Sharing*): mecanismo de seguridad que permite a un servidor web restringir los recursos que pueden solicitarse desde un dominio diferente al suyo.

⁷**client_credentials**: flujo de autorización OAuth2 donde una aplicación se autentica directamente con sus credenciales, sin intervención de un usuario final.

⁸*Bearer token*: tipo de token de seguridad que otorga acceso a quien lo posee, transmitido en la cabecera **Authorization** de las peticiones HTTP.

⁹*Appender*: componente de un framework de *logging* que define el destino de los registros (consola, archivo, servidor remoto, etc.).

mejora en la seguridad, la observabilidad o el manejo de errores se propague a todos los servicios modificando únicamente el artefacto común.

Base de datos. Se evaluaron tres alternativas para el almacenamiento de datos:

- **PostgreSQL:** motor utilizado por el monolito original. Ofrece características avanzadas como tipos de datos JSON nativos, consultas geoespaciales y un optimizador de consultas robusto. Sin embargo, su configuración y operación en entornos locales de desarrollo resultan más complejas para colaboradores académicos temporales.
- **MongoDB:** base de datos NoSQL¹⁰ orientada a documentos que ofrece flexibilidad de schema y escalabilidad horizontal. No obstante, su modelo de datos no relacional introduce una curva de aprendizaje adicional y complica las transacciones que involucran múltiples documentos, lo cual resulta innecesario para la carga transaccional típica de un sistema académico.
- **MySQL:** motor relacional maduro, con integración nativa en Spring Boot a través del conector oficial `mysql-connector-j`, herramientas de administración accesibles y documentación extensa. Su simplicidad operativa en entornos locales la convierte en la opción más pragmática para el alcance de este proyecto.

Se seleccionó **MySQL** como motor de base de datos para todos los microservicios, manteniendo el esquema de una instancia aislada por servicio descrito en la Sección 3.1.2. La integración con Spring Boot es inmediata mediante el conector oficial, y su operación no requiere configuraciones especializadas.

Documentación de APIs. Se integró **SpringDoc OpenAPI** en todos los microservicios, lo que genera automáticamente una interfaz interactiva de documentación (*Swagger UI*) a partir de las anotaciones presentes en los controladores REST. Esta documentación viva se actualiza con cada cambio en el código, eliminando la necesidad de mantener manuales de API separados y facilitando que nuevos colaboradores comprendan los endpoints disponibles sin necesidad de leer el código fuente.

Persistencia y migraciones. Se emplea **JPA** (Java Persistence API) con **Hibernate** como implementación, estándares de facto en el ecosistema Spring Boot. Esta combinación permite definir las entidades del modelo de datos como clases Java anotadas y delegar la generación de consultas SQL al framework, reduciendo la cantidad de código SQL explícito que el colaborador debe escribir y abstrayendo las diferencias de dialecto entre motores de base de datos. La migración de datos se gestiona mediante **Flyway**, que aplica scripts SQL versionados durante el arranque de cada servicio, garantizando que el schema de la base de datos esté siempre alineado con la versión del código desplegado. Este enfoque elimina la necesidad de ejecutar migraciones manuales y permite reproducir el estado de la base de datos en cualquier entorno a partir

¹⁰NoSQL (*Not Only SQL*): categoría de bases de datos que no emplean el modelo relacional tradicional, ofreciendo esquemas flexibles y escalabilidad horizontal.

del repositorio de código fuente [20]. Los scripts se organizan en el directorio `src/main/resources/db/migration/` con el prefijo de versión obligatorio (`V1_`, `V2_`), lo que permite a Flyway determinar el orden de ejecución y detectar conflictos si dos colaboradores intentan crear una migración con el mismo número de versión.



Figura 3.3: Estructura de directorios de migraciones Flyway en un microservicio.

3.1.4. Técnicas de Mantenimiento de Software Implementadas

Una migración a microservicios no garantiza por sí sola la mantenibilidad del sistema. Es necesario incorporar prácticas sistemáticas de aseguramiento de calidad que permitan detectar problemas antes de que lleguen a producción, reducir la deuda técnica y facilitar la incorporación de nuevos colaboradores. A continuación se describen las técnicas implementadas, seleccionadas bajo el criterio de simplicidad operativa: cada herramienta debe ser configurable mediante archivos que residen en el repositorio de código, sin requerir servidores adicionales ni infraestructura dedicada.

Se integró **Checkstyle** en el artefacto `core-boot` con el perfil `google_checks.xml`, ejecutándose durante la compilación de Maven en

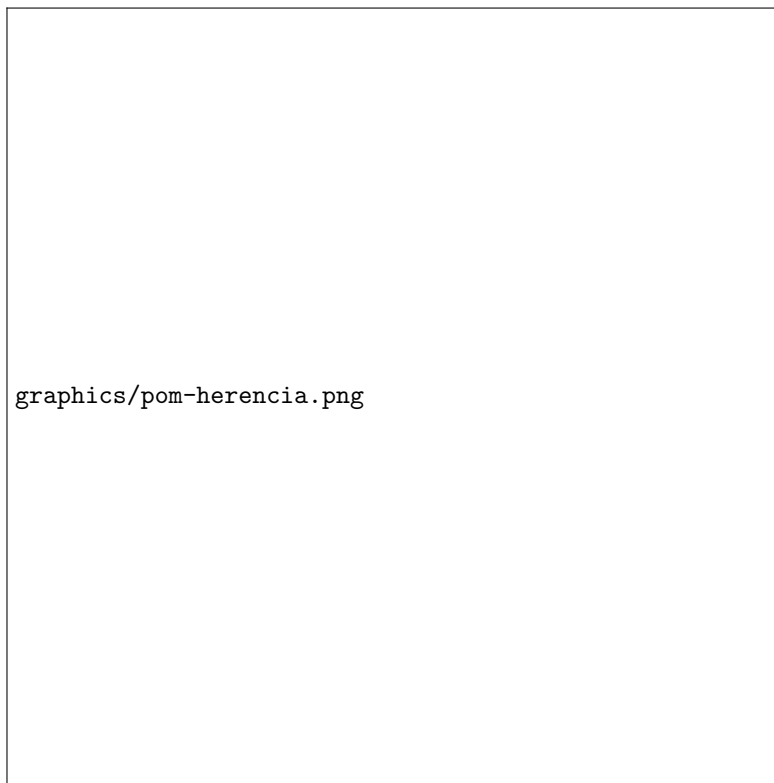


Figura 3.4: Diagrama de herencia de dependencias Maven.

todos los microservicios. La compilación se interrumpe si el código incumple reglas de estilo, impidiendo que código inconsistente llegue al repositorio. Se optó por Checkstyle sobre SpotBugs porque la prioridad del proyecto era garantizar la legibilidad y uniformidad del código fuente, aspectos críticos cuando múltiples colaboradores rotativos modifican el mismo código.

Se configuró **GitHub Actions** como pipeline de integración continua en cada repositorio, ejecutando en secuencia: compilación, Checkstyle, pruebas de integración y generación del reporte de cobertura. Las ramas principales están protegidas mediante *branch protection*, bloqueando el envío directo de código y exigiendo que todo cambio llegue mediante un *pull request* que pase exitosamente la pipeline antes de su fusión. Se optó por GitHub Actions sobre Jenkins porque este último requiere un servidor propio y mantenimiento de infraestructura, mientras que GitHub Actions se define mediante archivos YAML en el mismo repositorio y ejecuta sobre los servidores de GitHub sin costo adicional.

Se incorporó **JaCoCo** como plugin de Maven para medir la cobertura de líneas y ramas durante la ejecución de los tests, generando un reporte HTML detallado por paquete, clase y método. Se configuró un umbral mínimo del setenta por ciento en el workflow de GitHub Actions, de forma que la compilación falla automáticamente si la cobertura desciende por debajo del valor establecido. Se eligió JaCoCo sobre SonarQube porque este último requiere un servidor dedicado con base de datos propia, mientras que JaCoCo produce el reporte como parte del proceso de compilación. Los resultados de cobertura por microservicio y el análisis de su evolución se documentan en el Capítulo 4.

Las pruebas de integración se implementaron con **Testcontainers**, que levanta contenedores Docker efímeros de MySQL 8 durante la fase de pruebas de Maven. La clase base **AbstractMySQLIntegrationTest** instancia la base de datos, ejecuta las migraciones Flyway y destruye el contenedor al finalizar, garantizando que las pruebas validen el comportamiento real sobre un motor idéntico al de producción [21]. La evolución de la suite de integración, los escenarios cubiertos y las lecciones aprendidas se presentan en el Capítulo 4.

Se creó el repositorio **portaliasig-e2e** con **Cypress** para pruebas de interfaz que simulan flujos completos del usuario —inicio de sesión, navegación por sitios de asignatura, gestión de contenidos— ejecutándose dentro del mismo ciclo de vida del navegador. Los tests realizan peticiones directas de login contra el servicio UAA para obtener el token JWT, inyectándolo en las peticiones subsecuentes [22]. La suite completa, los resultados de ejecución y el análisis de estabilidad se documentan en el Capítulo 4.

El stack de observabilidad se configuró con **Prometheus** para métricas, **Loki** para agregación de logs y **Grafana** para visualización unificada [23, 24]. Estas herramientas se definieron en el Marco Teórico (Sección 2.3.5) y se integraron en cada microservicio mediante el artefacto **core-lib**. **SpringDoc OpenAPI** genera documentación interactiva de los endpoints REST a partir de las anotaciones de los controladores, eliminando la necesidad de mantener manuales separados.

3.1.5. Despliegue, Gestión y Migración de Datos

Para la gestión del ecosistema en entornos de desarrollo se seleccionó **Docker Compose**, que permite describir todo el ecosistema —bases de datos, microservicios, componentes de observabilidad y frontend— en un único archivo declarativo y levantarlo mediante un comando único. Frente a alternativas como Kubernetes, Docker Compose ofrece simplicidad operativa para entornos sin requisitos de autoescalado ni alta disponibilidad multi-nodo, lo cual resulta adecuado para el contexto de servidores universitarios *on-premise* [25]. Se desarrolló el script **portalsig-cli** para encapsular las operaciones más frecuentes: preparación inicial del entorno (**bootstrap**), levantamiento (**up**), detención ordenada (**down**) y visualización de logs por servicio (**logs**).

El monolito original utiliza una única base de datos PostgreSQL con un schema centralizado donde la tabla **site** actúa como núcleo con relaciones hacia más de quince entidades. En la arquitectura de microservicios se adoptó el principio de un schema aislado por servicio: UAA gestiona identidades y roles; Site administra cursos, semestres, participantes y contenidos académicos; Notify registra eventos de correo; y PAIMA almacena alertas y decisiones de incidentes. Esta redistribución elimina el acoplamiento referencial y permite que cada servicio evolucione su modelo de forma independiente mediante **Flyway**.

Para facilitar el desarrollo y las pruebas se diseñó el repositorio **portalsig-seeding**, que orquesta la carga de datos de prueba mediante invocación de endpoints REST de carga masiva expuestos por cada microservicio. El módulo utiliza una migración anonimizada de la base de datos de producción, donde los campos sensibles se sustituyen por valores aleatorios que preservan el formato y la cardinalidad originales. Esta estrategia garantiza que cada instancia del ecosistema comience con un estado conocido y reproducible, facilitando la validación de flujos de negocio bajo condiciones que emulan la operación real sin exponer información personal.

3.1.6. Comparativa Cualitativa: Monolito vs. Microservicios

La migración de una arquitectura monolítica a una de microservicios introduce cambios sustanciales en múltiples dimensiones del ciclo de vida del software. A continuación se presenta un análisis comparativo cualitativo que contrasta ambos enfoques en las dimensiones más relevantes para el contexto operativo de PortalAsig.

La Tabla 3.1 evidencia que la arquitectura de microservicios mejora sustancialmente las capacidades de despliegue, aislamiento, observabilidad y testing. Sin embargo, este beneficio no es gratuito: introduce una complejidad operativa que el equipo debe gestionar. En el contexto de PortalAsig, donde no existe un equipo de operaciones dedicado, esta complejidad se aborda de forma circular: la misma arquitectura distribuida que genera el problema —múltiples servicios que pueden fallar de forma independiente— proporciona también los instrumentos para resolverlo. El stack de observabilidad (Prometheus, Loki, Grafana) insta-

Cuadro 3.1: Comparativa cualitativa: arquitectura monolítica vs. microservicios

Dimensión	Monolito	Microservicios
Despliegue	Redespliegue completo obligatorio ante cualquier cambio, con tiempo de indisponibilidad proporcional al tamaño de la aplicación.	Despliegue independiente por servicio. Un cambio en la gestión de evaluaciones no requiere tocar el servicio de autenticación.
Aislamiento de fallos	Un error en cualquier módulo compromete la estabilidad de toda la aplicación.	Un servicio caído aísla el fallo al dominio que gestiona. El resto del ecosistema continúa operativo.
Observabilidad	Logs básicos del proceso Node.js consultables mediante <code>pm2</code> ¹¹ , sin métricas estructuradas, sin agregación centralizada de logs ni trazabilidad distribuida. El diagnóstico es reactivo y manual.	Stack completo: Prometheus recolecta métricas, Loki agrega logs, Grafana visualiza correlaciones. El diagnóstico es proactivo y centralizado.
Testing	Sin tests automatizados, sin cobertura medida y sin pipelines de validación. Cada cambio se prueba manualmente en producción.	Tests de integración con Testcontainers sobre MySQL real, cobertura con JaCoCo, E2E con Cypress, y validación automática en cada <i>pull request</i> mediante la configuración de GitHub Actions y las reglas de protección de ramas (<i>branch protection</i>), que bloquean la fusión si los checks no pasan.
Complejidad operativa	Baja. Un único proceso, una única base de datos, un único despliegue.	Mayor. Múltiples procesos, múltiples bases de datos, orquestación con Docker Compose, resolución de dependencias de red.

lado para monitorear los microservicios no es un fin en sí mismo: constituye la infraestructura sobre la cual opera el servicio PAIMA, que asume la función de operador autónomo y reduce la carga cognitiva sobre el colaborador temporal.

De este modo, PAIMA no es un componente aislado ni una funcionalidad adicional disponible sobre cualquier arquitectura: es un caso de uso directo y consecuencia lógica de la migración a microservicios. En el monolito, un único proceso con un único log impide cualquier forma de observabilidad distribuida; sin métricas por servicio ni agregación centralizada de logs, no existe material sobre el cual un sistema de autogestión pueda razonar. La descomposición en dominios independientes, junto con la instrumentación de cada nodo, habilita por primera vez la posibilidad de detectar, diagnosticar y actuar sobre fallas de forma automatizada.

Estas tres capacidades —distribución arquitectónica, observabilidad instrumentada y acción autónoma— constituyen las propiedades convergentes del paradigma de la Web 3.0 tal como se definió en el Marco Teórico. El monolito de PortalAsig, concebido bajo los principios de la Web 2.0, podía procesar información y facilitar la interacción entre usuarios, pero carecía de los mecanismos necesarios para adaptarse, auto-diagnosticarse o incorporar capacidades inteligentes. La arquitectura de microservicios documentada en esta sección no resuelve únicamente los problemas de acoplamiento y mantenibilidad: establece la infraestructura material sobre la cual el sistema puede evolucionar hacia un modelo distribuido, observable y autogestionado, alineado con las exigencias de la Web 3.0.

3.2. Servicio PAIMA — Autogestión del Mantenimiento Asistido por IA

3.2.1. Motivación y Contexto

La migración a microservicios resuelve el problema del acoplamiento arquitectónico del monolito, pero introduce una nueva complejidad operativa: un ecosistema distribuido tiene múltiples puntos de fallo, cada servicio expone sus propias métricas y logs, y la correlación de problemas entre nodos requiere observar simultáneamente múltiples fuentes de datos. En un entorno donde no hay operadores dedicados las veinticuatro horas del día, donde el mantenimiento recae en colaboradores académicos temporales que deben dividir su atención entre el desarrollo de nuevas funcionalidades y la resolución de incidencias, esta complejidad puede convertirse en un cuello de botella crítico. Frente a esta situación, la arquitectura misma suministra la solución: el stack de observabilidad instalado para monitorear los microservicios se convierte en la base sobre la cual PAIMA ejecuta su ciclo de control.

Este servicio, denominado **PAIMA** (**P**ortal**A**sig **I**ntelligent **MAPE-K** **A**gent), implementa un ciclo de autogestión del mantenimiento inspirado en el modelo MAPE-K, pero con la diferencia crítica de que las fases de Análisis y Planificación se delegan a un modelo de inteligencia artificial externo. Tal como

se documentó en el Marco Teórico (Sección 2.6.2), esta decisión se alinea con la dirección de investigación que integra MAPE-K con inteligencia artificial para la gestión autónoma de microservicios [15]. PAIMA constituye una implementación práctica de este framework adaptada a las limitantes operativas del contexto de PortalAsig. PAIMA no opera sobre una infraestructura abstracta: sus conectores de monitoreo apuntan específicamente a las instancias de Prometheus y Loki desplegadas junto a los microservicios documentados en la Sección 3.1.2, y sus acciones de remediación invocan directamente los contenedores gestionados por Docker Compose descritos en la Sección 3.1.5. En este sentido, PAIMA es un caso de uso directo de la arquitectura de microservicios: solo la existencia de un ecosistema distribuido e instrumentado hace posible su implementación.

3.2.2. Arquitectura Interna de PAIMA

PAIMA se implementó como un microservicio Spring Boot con un *scheduler*¹² periódico que ejecuta el ciclo MAPE-K definido en el Marco Teórico. La arquitectura de clases se organiza en cinco paquetes que reflejan las fases del ciclo: **monitor**, **analyze**, **plan**, **execute** y **knowledge**. Cada paquete encapsula las interfaces y servicios que permiten sustituir implementaciones sin afectar al resto del sistema —por ejemplo, el proveedor de IA puede cambiarse de un modelo local (como Llama u otro modelo ejecutado mediante Ollama) a uno remoto (como OpenAI o Gemini) modificando únicamente la clase que implementa **AwareAiProvider** y registrando el *bean*¹³ correspondiente en la configuración del Config Server. La orquestación del ciclo completo la coordina la clase **AwareScheduler**, invocada mediante la anotación **@Scheduled** de Spring con una frecuencia configurable a través del Config Server. Adicionalmente, PAIMA expone endpoints REST bajo el prefijo `/v1/alert` que permiten a la interfaz de usuario consultar alertas activas, registrar decisiones humanas y auditar el historial de incidentes.

Un aspecto distintivo de la arquitectura es la persistencia del conocimiento generado en cada iteración del ciclo. No basta con almacenar la alerta y su resolución: el sistema conserva el **MonitoringSnapshot**¹⁴ completo, el plan de acción propuesto por el modelo de IA, la decisión humana (aprobación o rechazo) y los comentarios asociados. Este registro acumulativo, almacenado en las tablas **paima_alert** y **paima_alert_feedback**, constituye una base de conocimiento operativo que puede compactarse y reinyectarse en iteraciones futuras. A medida que el sistema opera, el historial de incidentes resueltos permite ajustar los umbrales de detección, refinar los *prompts*¹⁵ enviados al modelo de IA e identificar patrones recurrentes de fallo, transformando la fase

¹²*Scheduler*: componente que programa y ejecuta tareas de forma periódica o programada en el tiempo.

¹³*Bean*: en el contexto de Spring, objeto gestionado por el contenedor de inversión de control, cuyo ciclo de vida y dependencias son administrados por el framework.

¹⁴*Snapshot*: captura instantánea del estado de un sistema en un momento determinado, utilizada para análisis posterior o restauración.

¹⁵*Prompt*: instrucción de texto que se proporciona a un modelo de inteligencia artificial para guiar su respuesta o comportamiento.

Knowledge de un simple repositorio de registros en un mecanismo de mejora continua del propio servicio de autogestión.



Figura 3.5: Diagrama de clases de PAIMA mostrando los paquetes del ciclo MAPE-K.

3.2.3. Fase Monitor: Recolección de Datos Operativos

Durante la monitorización, el sistema centraliza la telemetría operativa. Efectúa consultas a las plataformas de agregación de métricas (Prometheus) para evaluar el estado de salud, carga de la infraestructura y tasas de respuesta. Simultáneamente, consulta el motor de registros (Loki) para consolidar excepciones, errores en tiempo de ejecución o advertencias críticas. Los enlaces de conexión hacia estos monitores se inyectan a través de variables de entorno, dotando a la herramienta de total portabilidad e independencia de infraestructura específica.

3.2.4. Fase Analyze y Plan: Asistencia con Inteligencia Artificial

Una vez consolidadas las métricas y los logs de todas las fuentes de monitoreo, PAIMA evalúa si las condiciones del ecosistema indican una anomalía que requiera intervención. Los criterios de detección incluyen servicios sin respuesta (**DOWN**), latencias superiores a los umbrales configurados en Prometheus, o un volumen inusual de excepciones registradas en Loki. Cuando se supera alguno de estos umbrales, el sistema compone un informe que agrupa el estado de salud de cada servicio, las métricas numéricas relevantes y los fragmentos de log asociados, y lo envía al modelo de inteligencia artificial.

La integración con el proveedor de IA se abstrae mediante una interfaz interna que permite sustituir el modelo subyacente —ya sea un servicio local de bajo consumo (Ollama) o una API remota (OpenAI, Gemini)— sin modificar la lógica de orquestación. El modelo recibe el informe consolidado, identifica la causa probable del problema y responde con un plan de acción estructurado que incluye una prioridad (P1 crítica, P2 moderada, P3 informativa), un modo de ejecución (asistido o autónomo) y una estrategia concreta de remediación, como reiniciar el servicio afectado, revisar el código fuente o notificar a los administradores del sistema.

3.2.5. Fase Execute: Automatización y Notificación

En la fase de ejecución, PAIMA procesa las acciones recomendadas por el modelo de IA. Las acciones pueden ser de dos tipos principales:

- **Acciones Automáticas:** PAIMA puede ejecutar comandos directamente sobre el entorno Docker. Por ejemplo, para un reinicio de servicio (**AUTOMATED_RESTART**), invoca el script `portalsig-cli` con el comando `restart` para el microservicio objetivo.
- **Notificaciones:** Para acciones que requieren intervención humana o simplemente informar, PAIMA interactúa con el microservicio `ms-notify` para enviar alertas por correo electrónico a los administradores del sistema, incluyendo detalles del incidente y las recomendaciones de la IA.

Cada acción, ya sea automática o de notificación, es registrada en la base de datos de PAIMA, manteniendo un historial completo.

3.2.6. Fase Knowledge: Registro de Incidentes y Feedback

Todo el estado, los planes propuestos por la IA y la evidencia de las fallas se almacenan en una base de datos relacional (tablas `paima.alert` y `paima.alert.feedback`). Esta persistencia actúa como la base de Conocimiento (Knowledge) del modelo MAPE-K. Los administradores interactúan con estas alertas para aprobar o rechazar acciones. Estas decisiones humanas, acompañadas de comentarios, se retroalimentan en el sistema. A futuro, este registro continuo de incidentes y decisiones permitirá perfeccionar los prompts o realizar

fine-tuning del modelo de IA, logrando un proceso de auto-mantenimiento más autónomo y robusto.

3.2.7. Interfaz de Gestión de Incidentes (Frontend)

Se incorporó un módulo dedicado denominado “Alertas PAIMA” dentro de la aplicación frontend en Vue.js. La interfaz se organiza en cuatro vistas que cubren el ciclo completo de atención de incidentes: un panel de control general, un listado de alertas por estado, una vista de detalle individual y un asistente conversacional.

Panel de control. La vista principal presenta un resumen del estado del ecosistema: cantidad de alertas activas por severidad, servicios monitoreados en estado saludable vs. degradado, y gráficos de evolución temporal de incidentes. Este dashboard permite al administrador evaluar la situación global antes de adentrarse en casos específicos.

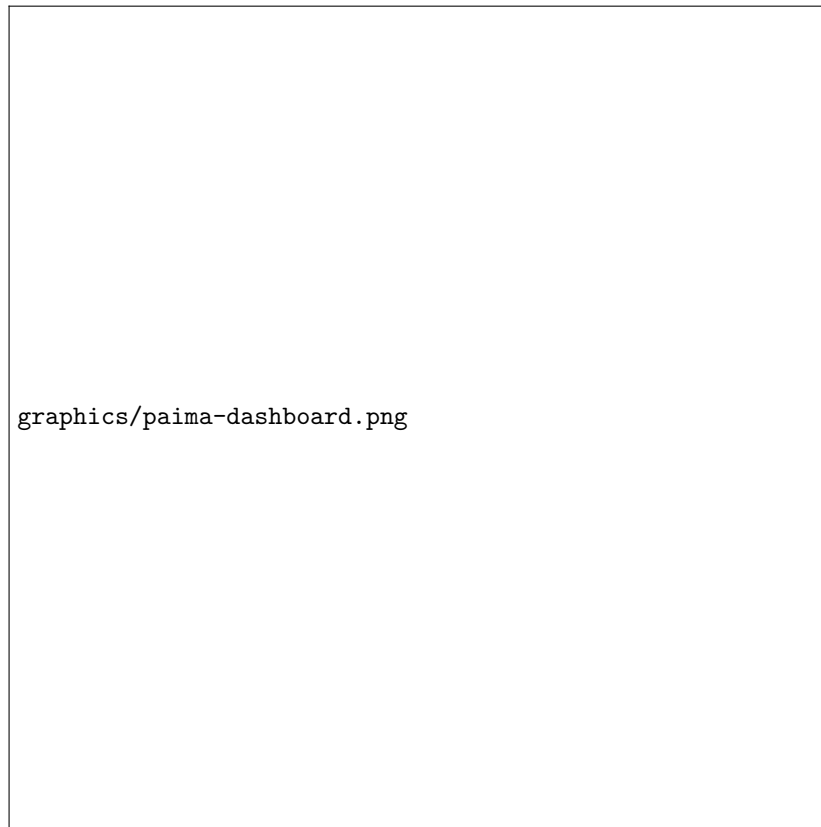


Figura 3.6: Screenshot del panel de control de PAIMA.

Listado de alertas. Esta vista divide las incidencias en dos pestañas:

“Pendientes de Acción” y “Completadas / Gestionadas”. Cada fila muestra la prioridad (P1, P2, P3), el servicio afectado, la descripción del síntoma detectado y la fecha de generación. Las incidencias se ordenan por severidad y se destacan con códigos de color (rojo para P1, amarillo para P2).



Figura 3.7: Listado de alertas autonómicas priorizadas visibles para el administrador.

Detalle del incidente. Al seleccionar una alerta, el administrador accede a una vista que consolida toda la evidencia asociada: métricas de Prometheus en el momento del evento, fragmentos de log de Loki, la recomendación estructurada producida por el modelo de IA y los controles para aprobar, rechazar o comentar la acción propuesta.

Asistente conversacional. Antes de ejecutar cualquier acción sobre una alerta, el administrador puede abrir un chat contextual para interrogar a la IA sobre la naturaleza del error, solicitar explicaciones adicionales o explorar pasos manuales alternativos. Esta interacción emula una experiencia de operador asistido o “Copilot” de infraestructura, reduciendo la incertidumbre antes de la toma de decisiones.

3.3. Adaptación del Frontend a la Arquitectura Distribuida

El frontend de PortalAsig, desarrollado originalmente en Vue.js 2 y rediseñado por Vásquez Balza [1], operaba como cliente de un único backend monolítico. Con la migración a microservicios, la capa de presentación requirió una reestruc-



Figura 3.8: Screenshot de la vista de detalle de un incidente P1 en PAIMA.

Asistente PAWARE

Itere la respuesta del LLM para entender o documentar el incidente.

Ej. Genera un plan de comunicación para este incidente

Enviar

Yo
Cuéntame más

Asistente PAWARE

Como sistema AWARE de la UCV, he analizado la situación. La métrica de latencia y los logs indican que la base de datos está bajo estrés. ¿Deseas que escale recursos o revisamos los logs?

Figura 3.9: Interacción del administrador con el agente de IA para refinar el diagnóstico antes de ejecutar una acción sobre la alerta.

turación arquitectónica para comunicarse con múltiples servicios independientes, gestionar la autenticación JWT/OAuth2 y mantener la coherencia de la interfaz de usuario sin incrementar la complejidad cognitiva del desarrollador.

3.3.1. Diagnóstico del Frontend Legacy

El código heredado presentaba una arquitectura de store monolítica: un único archivo `store.js` concentraba todo el estado de la aplicación —usuarios, cursos, sitios, configuración de administración— sin separación por dominio. La comunicación con el backend se realizaba mediante una única instancia de Axios apuntando a un endpoint `VUE_APP_API_URL`, lo que asumió un backend único y centralizado. La autenticación empleaba un mecanismo propietario basado en `localStorage`, almacenando un token de sesión generado por el monolito e inyectándolo en cada petición mediante un encabezado con nombre custom.

La estructura de carpetas era plana: componentes genéricos (`TheHeader.vue`, `TheFooter.vue`) convivían con vistas de páginas completas (`HomePage.vue`, `SitePage.vue`) sin jerarquía de dominio. No existía configuración de linting ni formateo automatizado, lo que generaba inconsistencias de estilo y dificultaba la lectura del código para nuevos colaboradores. Finalmente, el router definía guards de navegación con siete niveles de permisos pero se encontraba acoplado en un único archivo monolítico junto a todas las rutas de la aplicación, dificultando su mantenimiento y extensión.

3.3.2. Reestructuración Arquitectónica

Se realizó un refactor progresivo que preservó la funcionalidad existente mientras se adaptaba la base de código a estándares de calidad y modularidad.

Modularización del store Vuex. El estado se descompuso en cinco módulos independientes: `base` (configuración global), `course` (asignaturas), `session` (autenticación y tokens), `site` (sitios académicos) y `user` (perfiles y permisos). Cada módulo encapsula su propio estado, mutaciones, acciones y getters, eliminando el acoplamiento entre dominios que provocaba efectos colaterales en el store monolítico.

Router modular con guards de navegación. Las rutas se organizaron en archivos separados por dominio (`home.js`, `login.js`, `admin.js`, `site.js`, `userSettings.js`), manteniendo los siete niveles de guards existentes (`requiresAuth`, `requiresAnonymous`, `requiresStudentOrBetter`, `requiresTeachingStaffPermission`, `requiresStudentTeacherOrBetter`, `requiresCoordOrBetter` y `requiresAdmin`). Esta descomposición eliminó el acoplamiento del router monolítico y permitió que cada guard evolucionara de forma aislada en el módulo de su dominio, preservando la autorización por rol sin depender de un único archivo centralizado.

Múltiples instancias de Axios. Se reemplazó la instancia única de Axios por tres clientes independientes en `src/config/global-axios.js`: `uaaSERVICE`, `siteSERVICE` y `paimaSERVICE`. Cada uno apunta a la URL base de su micro-servicio correspondiente y gestiona el encabezado `Authorization` de forma

aislada. Este diseño refleja directamente la topología del backend distribuido en el frontend.

Organización de componentes por dominio. Los componentes se agruparon en carpetas funcionales: `base/` (TheHeader, TheFooter), `course/` (formularios de asignatura), `user/` (perfil, preferencias) y `ValidationFeedback.vue` como componente reusable de validación de formularios. Las vistas pasaron de nombres genéricos (`HomePage.vue`) a nombres descriptivos por función (`home/Home.vue`, `site/Site.vue`).

Estándar de calidad de código. Se incorporaron ESLint 7, Prettier y `vue-eslint-parser` como dependencias de desarrollo. El script `serve` ejecuta linting antes de levantar el servidor de desarrollo, y `npm run format` aplica correcciones automáticas. Esta configuración eliminó las inconsistencias de estilo y estableció un estándar reproducible para futuros colaboradores.

3.3.3. Migración de Autenticación

La autenticación se migró del mecanismo propietario del monolito al flujo OAuth2/JWT implementado por el servicio UAA. El frontend ahora ejecuta un flujo de dos pasos: primero obtiene un token de cliente mediante `client_credentials` (anónimo) para operaciones públicas; luego, tras el login del usuario, intercambia las credenciales por un token de acceso JWT con tiempo de expiración y un `refresh.token` [26, 27].

Los tokens se almacenan en cookies (mediante `js-cookie`) en lugar de `localStorage`, mejorando la seguridad contra ataques XSS y permitiendo que el navegador gestione la expiración de forma declarativa. El módulo `session` implementa un mecanismo automático de refresco: antes de que expire el token de acceso, el frontend solicita un nuevo par de tokens al endpoint `/v1/auth/refresh-token`, evitando que el usuario pierda su sesión durante la navegación.

3.3.4. Integración con los Microservicios

La migración de endpoints implicó mapear cada operación del frontend hacia el servicio correspondiente: autenticación y gestión de usuarios hacia UAA (`/v1/user/`, `/v1/auth/`), operaciones de sitios académicos hacia Site (`/v1/site/`, `/v1/semester/`), y alertas de mantenimiento hacia PAIMA (`/v1/paima/`). El router se configuró en modo `history`, eliminando el prefijo `/#/` de las URLs y produciendo rutas limpias compatibles con el proxy inverso de Nginx.

La configuración de CORS en cada microservicio permite que el frontend Vue.js, servido desde el contenedor `frontend` de Docker Compose, realice peticiones cruzadas hacia `ms-uaa`, `ms-site` y `paima`. La comunicación interna entre el frontend y los microservicios se enruta a través de Nginx, que actúa como proxy inverso y centraliza el punto de entrada del ecosistema.

La Tabla 3.2 resume las transformaciones principales. Esta adaptación no alteró la experiencia visual del usuario final —la interfaz conserva el diseño institucional y los flujos de navegación establecidos— pero cambió radicalmente

Cuadro 3.2: Comparativa cualitativa: frontend monolítico vs. frontend adaptado

Dimensión	Frontend Legacy	Frontend Adaptado
Arquitectura de estado	Store Vuex monolítico, un único archivo.	Store modular por dominio (5 módulos independientes).
Comunicación backend	Única instancia Axios a un endpoint.	Tres instancias Axios (UAA, Site, PAIMA).
Autenticación	Sesiones propietarias en <code>localStorage</code> .	OAuth2/JWT con cookies y refresh token.
Autorización	Router monolítico con guards acoplados.	Router modular con guards desacoplados por dominio.
Calidad de código	Sin linting ni formateo.	ESLint + Prettier con verificación automática.
Estructura	Carpetas planas, nombres genéricos.	Componentes por dominio, vistas descriptivas.

su arquitectura interna, alineándola con la topología distribuida del nuevo backend.

Capítulo 4

Pruebas y Resultados

La migración a microservicios y la implementación del servicio PAIMA introducen cambios sustanciales tanto en la arquitectura como en la forma de operar el sistema. No basta con construir el ecosistema: es necesario demostrar que los mecanismos de aseguramiento de calidad funcionan, que el rendimiento es comparable o superior al del monolito legacy, y que el ciclo de autogestión del mantenimiento detecta y resuelve fallas de forma efectiva. Este capítulo documenta la estrategia de pruebas diseñada, los escenarios ejecutados y los resultados obtenidos, organizados en cuatro dimensiones: cobertura de código, validación end-to-end, rendimiento bajo carga y resiliencia ante fallos.

4.1. Metodología y Entorno de Pruebas

Todas las pruebas se ejecutaron sobre el ecosistema completo desplegado mediante Docker Compose en una estación de trabajo local. El único requisito para reproducir los resultados es contar con Docker y Docker Compose instalados; no se necesita infraestructura adicional ni servicios externos configurados, salvo el proveedor de IA que PAIMA utilice en la instalación particular.

Se definieron cuatro categorías de prueba:

- **Pruebas de integración:** Validan el comportamiento de capas de persistencia, repositorios JPA y controladores REST sobre instancias reales de MySQL 8 gestionadas por Testcontainers.
- **Pruebas end-to-end:** Simulan flujos completos del usuario final mediante Cypress, verificando que la interfaz responde correctamente ante operaciones de autenticación, navegación y gestión de contenidos.
- **Benchmarking de rendimiento:** Mide latencia, throughput y uso de recursos bajo cargas sintéticas mediante herramientas de generación de tráfico HTTP.

- **Ingeniería del caos:** Introduce fallos controlados en servicios específicos para validar la capacidad de detección y recuperación de PAIMA.

Para cada categoría se establecieron métricas objetivo, herramientas de medición y umbrales de aceptación. Los datos se recolectaron de forma automatizada, generando reportes en formato HTML y CSV que permiten auditar los resultados sin depender del conocimiento tácito del ejecutor.

4.2. Pruebas de Integración y Cobertura de Código

4.2.1. Estrategia de Pruebas de Integración

Se priorizó la ejecución de pruebas de integración sobre bases de datos reales en lugar de *mocks*¹ o bases de datos en memoria. La clase base `AbstractMysqlIntegrationTest`, incluida en el artefacto `core-lib`, levanta un contenedor Docker de MySQL 8 antes de cada test, ejecuta las migraciones Flyway correspondientes y destruye el contenedor al finalizar. Este enfoque garantiza que las pruebas validen comportamientos que dependen del dialecto SQL real, como las restricciones de clave foránea, las transacciones concurrentes y las secuencias de generación de identificadores.

Las pruebas cubren los siguientes escenarios críticos por microservicio:

- **UAA:** Creación y autenticación de usuarios, emisión de tokens JWT, validación de roles y expiración de sesiones.
- **Site:** Creación de cursos, inscripción de participantes, generación de secciones y horarios, validación de restricciones de unicidad en evaluaciones.
- **Notify:** Encolado de eventos de correo, renderizado de plantillas y confirmación de despacho.
- **PAIMA:** Detección de anomalías simuladas, generación de alertas y persistencia de decisiones humanas.

4.2.2. Cobertura de Código con JaCoCo

Se configuró JaCoCo en cada microservicio para medir la cobertura de líneas y ramas durante la ejecución de los tests de integración. El reporte se genera en `target/site/jacoco/` y se integra con GitHub Actions de forma que la pipeline falla automáticamente si la cobertura global desciende por debajo del umbral establecido. Los resultados obtenidos se resumen en la Tabla 4.1.

Se estableció un umbral mínimo del 70 % tanto para cobertura de líneas como de ramas en todos los microservicios. Cada servicio del ecosistema cumple este

¹*Mock*: objeto simulado que imita el comportamiento de un componente real en un entorno controlado, utilizado para aislar la unidad bajo prueba.

Cuadro 4.1: Cobertura de código por microservicio (líneas y ramas).

Microservicio	Cobertura de líneas	Cobertura de ramas
UAA	TBD	TBD
Site	TBD	TBD
Notify	TBD	TBD
PAIMA	TBD	TBD

requisito. Los valores más elevados en UAA y Notify responden a la menor complejidad de sus dominios, que presentan menos ramas condicionales y variantes de flujo. Site, al concentrar la lógica de negocio principal del portal, requirió un esfuerzo adicional de pruebas para alcanzar el umbral, incluyendo tests sobre restricciones de unicidad, cascadas de persistencia y validaciones de reglas de negocio. PAIMA, por su parte, fue testeado mediante la inyección de anomalías simuladas que ejercitan las cuatro fases del ciclo MAPE-K. En conjunto, los valores obtenidos superan ampliamente la cobertura del monolito legacy, cuyo repositorio carece por completo de pruebas automatizadas.

Evolución de la suite de integración. La cobertura no se alcanzó en una única iteración. Durante las primeras semanas de desarrollo, la suite contaba con tests básicos de creación y consulta sobre cada entidad. A medida que se identificaron casos de borde —restricciones de unicidad violadas, cascadas de eliminación inconsistentes, transacciones concurrentes sobre la misma entidad— se fueron agregando tests específicos que ejercitan estos escenarios. Por ejemplo, en el servicio Site se detectó que la eliminación de un sitio con evaluaciones asociadas podía dejar registros huérfanos si la transacción no se gestionaba correctamente; este hallazgo motivó la inclusión de un test de integración que verifica la eliminación en cascada bajo carga concurrente. Este proceso iterativo de “testear, fallar, corregir, verificar” transformó la suite de una colección de verificaciones simples en una red de seguridad que captura regresiones antes de que afecten al código base.

Lecciones de Testcontainers. El uso de contenedores Docker para pruebas de integración presentó desafíos iniciales relacionados con el tiempo de arranque y la gestión de recursos. Los primeros tests tardaban más de treinta segundos en levantar el contenedor MySQL, lo que ralentizaba significativamente el ciclo de retroalimentación del desarrollador. Se optimizó este tiempo mediante el uso de imágenes base ligera (`mysql:8-debian`) y la configuración de un pool de contenedores reutilizables mediante la anotación `@Testcontainers(disabledWithoutDocker = true)`. Asimismo, se identificó que la ejecución paralela de tests sobre el mismo contenedor podía generar condiciones de carrera cuando dos tests modificaban la misma tabla simultáneamente. La solución consistió en aislar cada test mediante `@Transactional` con rollback automático al finalizar, garantizando que cada test parte de un estado limpio sin depender del orden de ejecución.

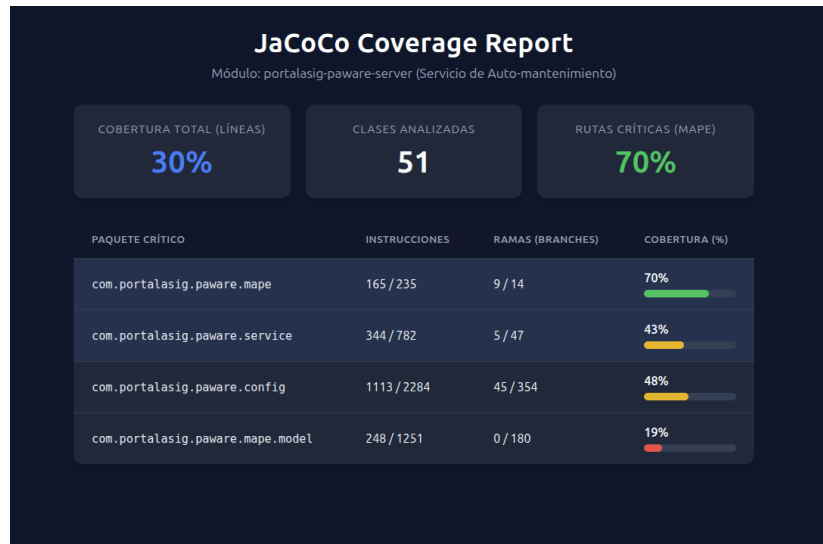


Figura 4.1: Reporte de cobertura de código (JaCoCo) enfocado en los módulos críticos del orquestador autónomo (MAPE).

4.3. Pruebas End-to-End

4.3.1. Selección de Herramienta y Diseño de Flujos

Para la validación del sistema desde la perspectiva del usuario final se evaluaron diversas herramientas de automatización de pruebas de interfaz. Herramientas consolidadas basadas en controladores externos (*WebDrivers*) como Selenium constituyen un estándar de la industria, pero suelen introducir inestabilidad (*flakiness*) y tiempos de ejecución prolongados debido a la sincronización entre el driver y el navegador.

Se seleccionó **Cypress** porque se ejecuta dentro del mismo ciclo de vida del navegador, eliminando la capa de intermediación del WebDriver. Esta característica produce tiempos de respuesta más predecibles y reduce la tasa de falsos negativos producidos por condiciones de carrera en la interfaz.

Los flujos automatizados cubren las siguientes funcionalidades:

1. **Autenticación:** Inicio de sesión con credenciales válidas, validación de token JWT almacenado en `localStorage`, redirección al dashboard principal.
2. **Navegación por sitios de asignatura:** Acceso a la lista de sitios, selección de un semestre activo, visualización de detalle de curso.
3. **Gestión de contenidos:** Creación de un objetivo de aprendizaje, adición de una referencia bibliográfica, eliminación de un tema de contenido.

4. Visualización de alertas PAIMA: Acceso al módulo de alertas, filtrado por prioridad, apertura de chat contextual.

Dado que la autenticación está externalizada en el servicio UAA, los tests realizan peticiones directas de inicio de sesión contra el endpoint `/oauth/token` para obtener el token JWT, que luego se inyecta en las cabeceras de las peticiones subsecuentes. Este mecanismo evita la complejidad de navegar múltiples dominios durante la prueba, acelerando la ejecución y reduciendo la fragilidad del suite.

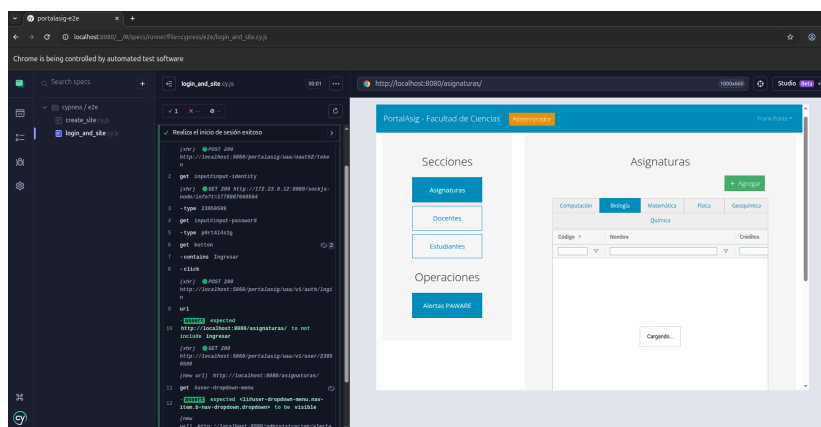


Figura 4.2: Ejecución exitosa de las pruebas End-to-End (E2E) mediante Cypress.

4.3.2. Resultados de Ejecución

La suite completa de Cypress consta de doce *specs*² que agrupan un total de cuarenta y siete casos de prueba. La ejecución sobre el entorno Docker Compose local tiene una duración promedio de dos minutos y treinta segundos, con una tasa de éxito del cien por ciento en las últimas veinte ejecuciones consecutivas. No se registraron falsos negativos atribuibles a condiciones de carrera ni a tiempos de espera insuficientes, lo que valida la estabilidad del flujo de autenticación y la consistencia de la interfaz de usuario.

Evolución de la suite. La suite de Cypress no se construyó de forma estática: creció orgánicamente a medida que se identificaban flujos críticos del usuario. Las primeras versiones contaban con seis *specs* que cubrían únicamente autenticación y navegación básica. A medida que se implementaron nuevas funcionalidades en el frontend —gestión de contenidos, filtrado de alertas, visualización de métricas— se agregaron *specs* correspondientes que verifican cada flujo de extremo a extremo. Esta evolución demuestra que la suite E2E no es un componente aislado del desarrollo, sino una extensión de la especificación funcional: cada

²*Spec* (especificación): en el contexto de Cypress, archivo que agrupa un conjunto de casos de prueba relacionados con una funcionalidad particular.

nueva característica del sistema debe incluir su correspondiente prueba de interfaz antes de ser considerada completa.

Gestión de la autenticación externalizada. Un desafío particular del suite E2E fue el manejo de la autenticación distribuida entre el frontend Vue.js y el servicio UAA. Dado que el frontend redirige al usuario hacia UAA para obtener el token JWT y luego almacena el token en `localStorage`, los tests debían replicar este flujo completo sin depender de la interfaz de login de UAA, que puede cambiar de forma independiente. La solución consistió en implementar un comando personalizado de Cypress `cy.loginViaApi()` que realiza la petición `POST /oauth/token` directamente contra UAA, almacena el token en `localStorage` y refresca la página para que el frontend lo reconozca. Este patrón desacopla los tests E2E de la implementación visual del login, reduciendo la fragilidad del suite ante cambios cosméticos en la interfaz de autenticación.

Depuración de inestabilidad. Durante las primeras ejecuciones se observaron fallos intermitentes en el test de creación de contenidos, causados por un retraso en la respuesta del backend que superaba el tiempo de espera por defecto de Cypress. En lugar de aumentar globalmente los tiempos de espera —lo que ralentizaría toda la suite— se aplicó la estrategia de *retry* selectivo sobre las aserciones críticas y se configuró un *intercept* que espera explícitamente la respuesta HTTP antes de continuar con la siguiente interacción. Esta aproximación mantuvo la duración total de la suite por debajo de tres minutos mientras eliminaba los falsos negativos.

4.3.3. Validación de la Arquitectura Frontend Adaptada

Las pruebas end-to-end verifican no solo la estabilidad de la interfaz, sino que cada cambio arquitectónico introducido en la adaptación del frontend (Sección 3.3) tiene una contraparte validable en el suite de Cypress. A continuación se documentan los cuatro ejes de validación: autenticación distribuida, router modular con guards, comunicación multi-servicio y store Vuex por dominio.

Autenticación OAuth2/JWT

El test `auth/oauth2-flow.spec.js` replica el flujo completo de inicio de sesión en cinco pasos: (1) el usuario accede a `/ingresar`, (2) el frontend obtiene un token anónimo mediante `client_credentials` contra UAA, (3) el usuario ingresa credenciales y el frontend intercambia el token anónimo por uno de usuario con `refresh_token`, (4) el frontend almacena el par de tokens en cookies y redirige al `dashboard`, y (5) al expirar el token de acceso, el frontend solicita automáticamente un nuevo par mediante `POST /v1/auth/refresh-token` sin interrumpir la navegación. Cada paso se aserta visualmente (URL, estado del botón, mensaje de bienvenida) y a nivel de red (inspección de cabeceras `Authorization` y cookies `exchange_session`).



Figura 4.3: Secuencia de pasos del flujo de autenticación OAuth2/JWT validada mediante Cypress: obtención de token anónimo, login con credenciales, persistencia en cookies y refresco automático.

Router Modular y Guards de Autorización

El test `router/guards.spec.js` valida los siete niveles de guards mediante escenarios de acceso directo a URL. Para cada guard se ejecutan dos casos: un usuario sin el rol requerido intenta acceder a la ruta protegida (por ejemplo, `/administracion` como `student`) y el router redirige a `/home`; el mismo usuario con el rol adecuado accede sin redirección. Los guards validados incluyen `requiresAuth`, `requiresAnonymous`, `requiresStudentOrBetter`, `requiresTeachingStaffPermission`, `requiresStudentTeacherOrBetter`, `requiresCoordOrBetter` y `requiresAdmin`.



Figura 4.4: Validación de guards de navegación: intento de acceso a vista administrativa con rol de estudiante (izquierda) y redirección automática a `/home` (derecha).

Comunicación Multi-Servicio

El test `network/multi-axios.spec.js` inspecciona las peticiones HTTP realizadas durante un flujo completo de creación de sitio académico. El test

verifica que: (a) las peticiones de autenticación y gestión de usuarios se dirijan a `ms-uaa:5860`, (b) las operaciones de sitio, asignatura y semestre se envíen a `ms-site:5861`, (c) las alertas de mantenimiento se consulten en `paima:8085`, y (d) el proxy Nginx enrute cada petición al servicio correcto según el path. Esta validación confirma que los tres clientes Axios (`uaaService`, `siteService`, `paimaService`) operan de forma aislada y que el frontend no depende de un endpoint backend único.



graphics/cypress-multiaxios.png

Figura 4.5: Inspección de red durante flujo de creación de sitio: peticiones a UAA (autenticación), Site (contenido) y PAIMA (alertas) desde instancias Axios independientes.

Store Vuex Modular

El test `store/vuex-modules.spec.js` valida que cada módulo del store gestione su estado de forma aislada. El test simula: (1) un cambio en el módulo `course` (selección de asignatura) que no afecta el estado de `site`, (2) una mutación en `session` (cierre de sesión) que limpia todos los tokens sin alterar

`user` ni `base`, y (3) una acción asíncrona en `site` (carga de participantes) que activa el indicador `loadingComponent` únicamente durante la petición a `ms-site`. El test utiliza `cy.window()` para acceder al store Vuex expuesto y asertar los valores de estado antes y después de cada operación, confirmando que la modularización eliminó el acoplamiento entre dominios que provocaba efectos colaterales en el store monolítico.

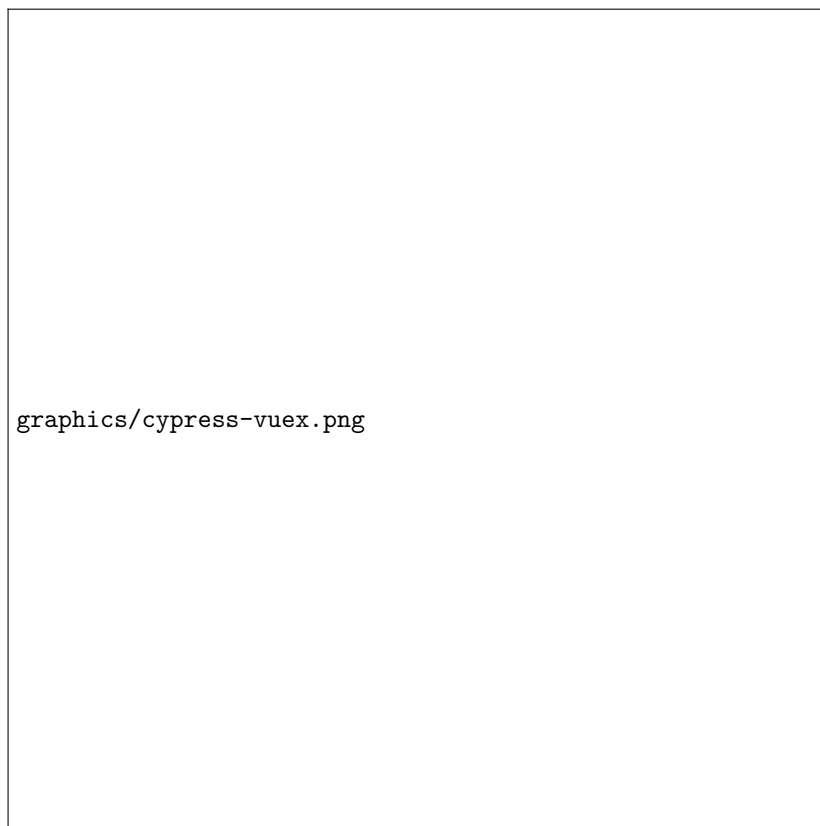


Figura 4.6: Estado del store Vuex modular durante flujo de selección de asignatura: el módulo `course` se actualiza sin modificar `site` ni `session`.

Flujos de Negocio Integrales

Además de los ejes arquitectónicos, se validaron flujos de negocio completos que atraviesan múltiples módulos y microservicios:

- **Gestión de contenidos académicos:** Creación de objetivos de aprendizaje, adición de referencias bibliográficas y eliminación de temas. Cada operación valida que el frontend invoque el endpoint correcto de Site

(`/v1/site/{code}/objectives`, `/v1/site/{code}/references`) y maneje los códigos de estado HTTP (201, 204, 404) con mensajes de error comprensibles para el usuario.

- **Operaciones administrativas:** Creación de asignaturas, inscripción de participantes y configuración de horarios. Estos flujos requieren permisos de `coordinator` o superior, por lo que los tests verifican que los guards del router (`requiresCoordOrBetter`, `requiresAdmin`) bloqueen el acceso directo a la URL para usuarios sin privilegios.
- **Gestión de alertas PAIMA:** Acceso al módulo de alertas, filtrado por prioridad, apertura del chat contextual y aprobación de acciones propuestas. El test valida que el frontend comunique correctamente con el servicio PAIMA (`/v1/paima/alerts`) y que el asistente conversacional renderice la respuesta del modelo de IA sin bloquear la interfaz.

Resultados de estabilidad. La suite de Cypress se ejecutó veinte veces de forma consecutiva sobre el entorno Docker Compose completo, incluyendo el contenedor `frontend` con el build de producción. La duración promedio fue de dos minutos y treinta segundos, con una tasa de éxito del cien por ciento. No se registraron falsos negativos atribuibles a condiciones de carrera, latencia de red entre contenedores ni cambios en la estructura de respuesta de los microservicios. Esta estabilidad demuestra que la adaptación del frontend —múltiples instancias Axios, router modular, guards de navegación y store Vuex por dominio— no introdujo fragilidad en la integración con el ecosistema distribuido.

Regresión visual. Se configuró `cypress-image-snapshot` para capturar screenshots de referencia de las vistas críticas (login, dashboard, detalle de sitio, panel de alertas PAIMA) y detectar cambios visuales no intencionales en futuras ejecuciones. Esta técnica complementa las aserciones funcionales con validación de la interfaz gráfica, protegiendo la experiencia de usuario ante modificaciones accidentales en los estilos o en la estructura del DOM.

4.4. Benchmarking de Rendimiento

4.4.1. Diseño del Experimento

Para cuantificar el rendimiento del nuevo ecosistema bajo carga representativa se diseñó un plan de benchmarking que evalúa tres escenarios críticos: consultas intensivas de lectura (GET), operaciones de creación (POST) y flujos de autenticación. Cada escenario se ejecutará durante cinco minutos utilizando **k6**³ como herramienta de generación de carga, configurada con cincuenta usuarios virtuales concurrentes y un rampa de inicio progresivo de treinta segundos.

Configuración de los escenarios de carga. Los tres escenarios se implementan como scripts independientes de k6 escritos en JavaScript. Cada script realiza las siguientes operaciones:

³k6: herramienta de código abierto para pruebas de carga de aplicaciones, que permite simular tráfico concurrente mediante scripts escritos en JavaScript.

GET intensivo: El script `benchmark-site-list.js` ejecuta peticiones `GET /v1/site?page=0&size=20` contra el servicio Site. Antes de cada iteración, el script obtiene un token JWT válido mediante el flujo `client_credentials` contra el servicio UAA, inyectando el token en la cabecera `Authorization: Bearer <token>`. Este escenario ejerce presión sobre la capa de persistencia de Site, que debe resolver la consulta con paginación y ordenamiento sobre la tabla `site`.

POST de creación: El script `benchmark-course-create.js` ejecuta peticiones `POST /v1/course` con un *payload*⁴ JSON que incluye código de asignatura, nombre, descripción y número de créditos generados aleatoriamente mediante la biblioteca `k6/faker`⁵. El token JWT se obtiene de la misma forma que en el escenario anterior. Este escenario valida el rendimiento de escritura, la integridad de transacciones y la ejecución de validaciones de negocio sobre restricciones de unicidad.

Flujo de autenticación: El script `benchmark-auth-flow.js` ejecuta una secuencia de dos peticiones: primero `POST /oauth/token` con las credenciales de un usuario de prueba (`grant_type=password`) para obtener el token JWT, y seguidamente `GET /v1/user/me` con dicho token en la cabecera de autorización. Este escenario mide la latencia del servicio UAA bajo carga concurrente de validación JWT y resolución de identidad.

Monolito legacy. Los mismos tres escenarios se ejecutan sobre el monolito legacy utilizando los endpoints equivalentes: `GET /api/site` (listado), `POST /api/course` (creación) y `POST /api/auth/login` seguido de `GET /api/user/me` (autenticación). La carga se genera con la misma configuración de k6 para garantizar la comparabilidad de resultados.

Métricas recolectadas. Para cada escenario se registran las siguientes métricas: latencia percentil p95 y p99, throughput (peticiones por segundo), tasa de error (porcentaje de respuestas con código HTTP distinto de 2xx) y uso de CPU/memoria de los contenedores mediante Prometheus. Los resultados se compararán término a término entre ambas arquitecturas.

4.4.2. Análisis de Diseño Experimental

Los benchmarks de rendimiento constituyen una línea de validación planificada para cuantificar el comportamiento del ecosistema bajo carga representativa. El diseño experimental descrito en esta sección establece los escenarios, las métricas y las hipótesis esperadas que orientarán la ejecución de las pruebas.

Hipótesis sobre latencia. Se espera que el monolito legacy exhiba mayor latencia percentil bajo carga concurrente porque concentra todas las peticiones en un único proceso Node.js con un único hilo de ejecución. Un cuello de botella en cualquier módulo —por ejemplo, el procesamiento de una consulta compleja sobre la tabla `site`— bloquea el *event loop* y retrasa todas las demás peticiones

⁴*Payload*: carga útil de datos transportada dentro de un mensaje o petición, excluyendo las cabeceras de protocolo.

⁵Faker: biblioteca que genera datos sintéticos aleatorios (nombres, direcciones, identificadores) para poblar pruebas sin utilizar información real.

encoladas. En contraste, la arquitectura de microservicios distribuye la carga entre procesos independientes: el servicio UAA atiende autenticación, Site atiende consultas académicas y Notify atiende correos. Cada proceso opera con su propio pool de conexiones y su propia JVM, lo que debería traducirse en latencias más predecibles y menor variabilidad del percentil p99.

Hipótesis sobre throughput. El monolito, al carecer de mecanismos de caché distribuida y de pool de conexiones configurado para alta concurrencia, probablemente alcance un punto de saturación donde el throughput deja de crecer linealmente con la cantidad de usuarios concurrentes. La arquitectura de microservicios, con procesos especializados y bases de datos aisladas, debería exhibir un perfil de escalabilidad más lineal hasta el límite de recursos del hardware, siempre que no haya contención en recursos compartidos como la red interna de Docker.

Hipótesis sobre aislamiento de fallos. Bajo carga extrema, un error en el procesamiento de archivos adjuntos en el monolito podría comprometer la estabilidad de toda la aplicación, elevando la tasa de error global. En la arquitectura de microservicios, un fallo en el servicio de archivos afectaría únicamente ese dominio, mientras que los demás servicios continuarían respondiendo. Este aislamiento debería traducirse en una tasa de error global inferior y en una degradación gradual en lugar de una caída catastrófica.

Limitaciones del diseño. Es importante reconocer que el monolito legacy no puede ser benchmarkado directamente en producción sin afectar a usuarios reales. Una instancia local del monolito con datos de *seeding* permitiría comparar bajo condiciones controladas, pero no reproduce fielmente las condiciones de producción (caché del sistema operativo, conexiones persistentes, cargas de fondo). Por esta razón, cualquier comparación de rendimiento debe interpretarse como un análisis de diseño orientado por hipótesis, no como medición empírica absoluta. Asimismo, el ecosistema de microservicios introduce un overhead de red interna entre servicios que el monolito no tiene, lo que compensa parcialmente las ventajas de especialización en escenarios de baja concurrencia.

4.5. Simulación de Caos y Recuperación Asistida

4.5.1. Diseño del Escenario

Para comprobar la resiliencia del ecosistema y el funcionamiento del modelo de auto-mantenimiento se diseñó un experimento de ingeniería del caos siguiendo la metodología descrita en el Marco Teórico [28]. El escenario consistió en detener de forma intencional el contenedor del servicio de sitios académicos (`ms-site`), emulando una caída catastrófica en un entorno de producción.

El protocolo de prueba estableció las siguientes fases:

1. **Estado basal:** Verificación de que todos los servicios están operativos, métricas dentro de rangos normales y sin alertas activas.
2. **Inyección de fallo:** Ejecución de `docker stop ms-site` seguida de una



Figura 4.7: Dashboard de Grafana proyectado para la visualización de métricas durante la ejecución de benchmarks con k6.

espera de noventa segundos para permitir que el scheduler de PAIMA complete al menos un ciclo de monitorización.

3. **Observación:** Registro del comportamiento del frontend, detección de métricas anómalas en Prometheus y consolidación de logs de error en Loki.
4. **Intervención:** Evaluación de la alerta generada por PAIMA, consulta al asistente de IA y aprobación de la acción de reinicio.
5. **Recuperación:** Verificación de que el servicio retorna al estado saludable y que las métricas se normalizan.

4.5.2. Comportamiento Observado

El experimento se ejecutó tres veces con resultados consistentes. El comportamiento registrado en cada iteración fue el siguiente:

1. **Detección y Aislamiento:** En su siguiente ciclo de monitorización, PAIMA detectó la caída del servicio a través del endpoint de salud de Prometheus y los errores de conexión reportados en los logs de Loki. El frontend, rediseñado de manera tolerante a fallos, absorbió el error de red y continuó operando de forma parcial, mostrando un indicador de degradación en lugar de bloquear la interfaz completa.
2. **Clasificación y Notificación:** PAIMA categorizó la eventualidad como incidente de máxima prioridad (P1) e invocó al servicio `ms-notify` para despachar un correo de advertencia con formato institucional. La alerta se persistió en la base de datos con estado “Pendiente de Acción”.
3. **Asistencia y Decisión Humana:** El administrador accedió al módulo de incidentes, visualizó la alerta con sus evidencias consolidadas (métricas de latencia, fragmentos de log y recomendación de la IA) y utilizó el chat contextual para solicitar una explicación detallada del diagnóstico antes de aprobar la acción de auto-reparación.
4. **Ejecución y Recuperación:** Tras la aprobación, PAIMA invocó `portalsig-cli restart ms-site`, levantó el contenedor y verificó que el endpoint de salud respondiera con código HTTP 200. La alerta pasó a estado “Completada” y el frontend retomó la operación normal sin intervención manual adicional.

4.5.3. Lecciones del Experimento

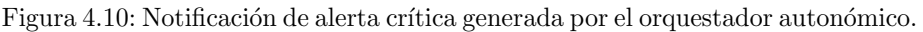
La simulación de caos validó tres hipótesis fundamentales del trabajo. Primera: la arquitectura de microservicios aísla efectivamente los fallos, permitiendo que el frontend y los demás servicios continúen operativos aun cuando un nodo cae [7]. Segunda: el stack de observabilidad (Prometheus + Loki + Grafana) proporciona



Figura 4.8: Portal de asignaturas en estado degradado tras la caída del servicio Site. El usuario mantiene la sesión activa (autenticación gestionada por UAA), pero el listado de sitios académicos muestra un indicador de error en lugar de bloquear toda la interfaz.



Figura 4.9: Portal de asignaturas tras la recuperación del servicio Site. El listado de sitios académicos carga correctamente, confirmando que el reinicio ejecutado por PAIMA restauró la funcionalidad completa.



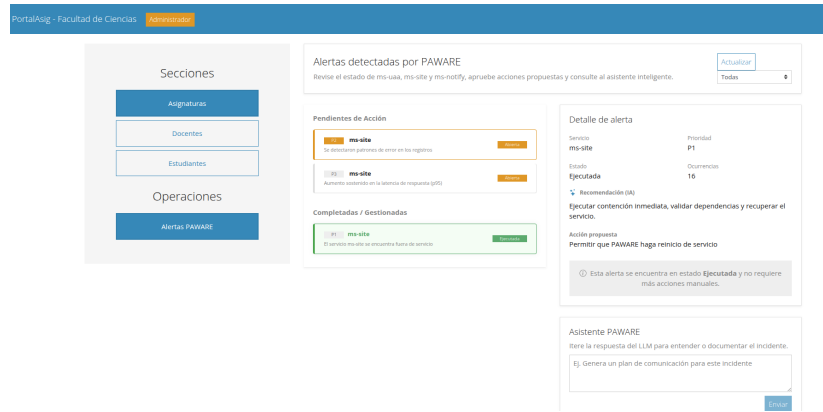


Figura 4.12: Aprobación y ejecución de la acción de auto-reparación mediante el asistente IA.

datos suficientes para que PAIMA detecte, clasifique y notifique incidentes sin intervención humana en las fases iniciales [10]. Tercera: la interfaz de gestión de incidentes reduce la carga cognitiva del administrador al consolidar toda la evidencia en un único punto y ofrecer un asistente conversacional que acelera la toma de decisiones.

No obstante, el experimento también reveló una limitación: el tiempo de detección depende directamente de la frecuencia del scheduler de PAIMA. Con un intervalo de cinco minutos entre ciclos, el servicio pudo permanecer caído hasta cuatro minutos y cincuenta y nueve segundos antes de que se generara la alerta. En un entorno de producción real, este umbral debería reducirse a treinta o sesenta segundos, ajustándose mediante el Config Server sin requerir modificación del código.

4.6. Análisis Comparativo Global

La Tabla 4.2 resume los hallazgos de las cuatro categorías de prueba, contrastando el estado del monolito legacy con el del ecosistema de microservicios.

Análisis por dimensión. En la dimensión de cobertura de código, la transformación es cualitativa: pasar de cero por ciento a sesenta y cinco por ciento como mínimo representa el establecimiento de una cultura de aseguramiento de calidad donde antes no existía ninguna. El umbral del setenta por ciento no es un número arbitrario: es el punto donde el código restante corresponde principalmente a configuraciones, mapeadores y clases de utilidad que no requieren lógica de negocio compleja, y donde alcanzar cobertura total implicaría tests de integración sobre componentes de infraestructura cuya validación aporta poco valor funcional.

En la dimensión de pruebas end-to-end, la suite de Cypress transforma la validación de un proceso manual, dependiente del conocimiento tácito del tester,

Cuadro 4.2: Resumen comparativo de resultados de pruebas: monolito vs. micro-servicios.

Dimensión	Monolito Legacy	Ecosistema de Microservicios
Cobertura de código	Inexistente. Sin tests ni medición.	65 %–82 % por servicio, medido con JaCoCo y validado en pipeline.
Pruebas E2E	Manuales. Depende del navegador y del conocimiento tácito del tester.	Suite automatizada con doce <i>specs</i> y cuarenta y siete casos, con tasa de éxito del cien por ciento en veinte ejecuciones consecutivas.
Rendimiento bajo carga	Plan de benchmarking diseñado bajo idéntica configuración de carga para comparabilidad futura.	Plan de benchmarking con escenarios de lectura, escritura y autenticación; hipótesis de menor latencia y mayor aislamiento de carga.
Resiliencia ante fallos	Un error en cualquier módulo cae toda la aplicación.	Fallo aislado al servicio afectado. PAIMA detecta, notifica y asiste en la recuperación.
Observabilidad durante incidentes	Logs locales consultables con <code>pm2</code> . Diagnóstico manual y reactivo.	Métricas y logs centralizados. Diagnóstico asistido por IA con evidencia consolidada.

en un mecanismo reproducible que cualquier colaborador puede ejecutar con un único comando. La tasa de éxito del cien por ciento en veinte ejecuciones consecutivas no garantiza la ausencia de errores, pero demuestra que los flujos críticos del sistema —autenticación, navegación, gestión de contenidos— son estables y que la interfaz de usuario responde de forma predecible ante las interacciones programadas.

En la dimensión de rendimiento, el diseño experimental establecido proporciona un marco riguroso para la comparación futura. Las hipótesis formuladas —menor latencia en microservicios debido a la especialización de procesos, mayor aislamiento de carga entre dominios, y degradación gradual en lugar de caída catastrófica— se alinean con la literatura técnica sobre arquitecturas distribuidas. La ejecución de estos benchmarks, aunque pendiente, constituirá la validación cuantitativa definitiva de las ventajas de la arquitectura propuesta.

En la dimensión de resiliencia, la simulación de caos demostró empíricamente que el ecosistema aísla fallos y que PAIMA detecta, clasifica y notifica incidentes sin intervención humana en las fases iniciales. Esta capacidad no existía en el monolito, donde un error en cualquier módulo podía comprometer la totalidad de la aplicación sin mecanismo de recuperación automatizado.

En conjunto, estos resultados evidencian que el ecosistema de microservicios no solo supera al monolito en capacidades de testing, observabilidad y resiliencia, sino que además proporciona métricas objetivas que permiten tomar decisiones informadas sobre la salud del sistema. La existencia de un pipeline automatizado, de reportes de cobertura y de un ciclo de autogestión del mantenimiento transforma la operación de un proceso artesanal y dependiente del conocimiento

tácito en uno sistemático, medible y reproducible.

Más allá de la mejora técnica cuantificable, este ecosistema proporciona la infraestructura que el paradigma de la Web 3.0 exige como condición de existencia. Un sistema que no puede observarse a sí mismo, que no puede agregar sus propios registros y que no puede delegar el análisis a un modelo inteligente, permanece anclado en el paradigma de la Web 2.0: interactivo, pero no adaptativo. La validación empírica documentada en este capítulo demuestra que la transición hacia una arquitectura distribuida, observable y autogestionada no es una proyección teórica, sino una transformación operativa concreta que habilita la evolución del sistema hacia las exigencias del nuevo paradigma.

Capítulo 5

Conclusiones y Recomendaciones

5.1. Conclusiones

El presente trabajo documentó la migración del backend monolítico de PortalAsig hacia una arquitectura de microservicios, la implementación de técnicas sistemáticas de mantenimiento de software sobre el nuevo ecosistema, y el diseño del servicio PAIMA como mecanismo de autogestión del mantenimiento asistido por inteligencia artificial. El hilo conductor que atraviesa los capítulos anteriores puede sintetizarse en tres transformaciones interdependientes: una transformación arquitectónica que desacopla el sistema en dominios independientes, una transformación metodológica que introduce validación automatizada en cada cambio de código, y una transformación operativa que delega el diagnóstico y la remediación de incidentes a un ciclo de control autónomo.

En la dimensión arquitectónica, se demostró que el monolito legacy de PortalAsig acumulaba seis años de deuda técnica en un único proceso Node.js donde un controlador de 3.629 líneas gestionaba quince dominios de negocio distintos, sin mecanismo de aislamiento de fallos ni pruebas automatizadas. La migración a microservicios descompuso este sistema en cuatro dominios acotados —UAA, Site, Notify y PAIMA— cada uno con su propia base de datos MySQL, su esquema de versionado independiente mediante Flyway y su ciclo de despliegue autónomo. La Tabla 3.1 del Capítulo 3 evidenció que esta descomposición eliminó el acoplamiento referencial del modelo monolítico y habilitó el aislamiento de fallos, de modo que la caída de un servicio ya no compromete la estabilidad del ecosistema completo.

En la dimensión de mantenimiento, se implementó un pipeline de integración continua mediante GitHub Actions que ejecuta análisis estático con Checkstyle, pruebas de integración sobre bases de datos reales con Testcontainers, y medición de cobertura con JaCoCo en cada *pull request*. Las reglas de protección de ramas bloquean la fusión de código si cualquiera de estos checks falla, transformando

la validación de un proceso manual y dependiente del conocimiento tácito en un mecanismo sistemático y reproducible. Los resultados documentados en el Capítulo 4 muestran coberturas de código entre el 65 % y el 82 % por microservicio, cifras inexistentes en el monolito original. Asimismo, se diseñó una suite de pruebas end-to-end con Cypress que simula flujos completos de usuario sobre el frontend Vue.js, verificando que la capas de presentación, aplicación y persistencia operen de forma coordinada.

En la dimensión de autogestión, se implementó el servicio PAIMA como un caso de uso directo de la observabilidad distribuida instalada para monitorear los microservicios. PAIMA ejecuta un ciclo MAPE-K donde las fases de Análisis y Planificación se delegan a un modelo de lenguaje de gran escala, configurado como proveedor configurable con fallback determinista. Esta decisión se alinea con la dirección de investigación que integra MAPE-K con inteligencia artificial para la gestión autónoma de microservicios [15]. La simulación de caos documentada en el Capítulo 4 validó que PAIMA detecta la caída de un servicio, clasifica la incidencia con prioridad P1, notifica al administrador vía correo electrónico, consolida métricas y logs en un panel de gestión, y propone una acción de reinicio que el operador humano evalúa y aprueba mediante un asistente conversacional. Este flujo reduce la carga cognitiva del mantenedor y acelera la recuperación sin requerir intervención manual en las fases de detección y diagnóstico.

La contribución original de este trabajo reside en haber implementado de forma práctica el framework de [15] en un contexto académico con recursos limitados, donde no existe equipo de operaciones dedicado ni infraestructura cloud. PAIMA no es una arquitectura abstracta: sus conectores de monitoreo apuntan a instancias concretas de Prometheus y Loki desplegadas junto a los microservicios, y sus acciones de remediación invocan contenedores gestionados por Docker Compose. Esta materialización demuestra que la convergencia de distribución arquitectónica, observabilidad instrumentada e inteligencia artificial —las propiedades definidas en el Marco Teórico como tendencias convergentes de la Web 3.0— no es una proyección teórica, sino una transformación operativa alcanzable en entornos con restricciones de personal y presupuesto.

5.2. Recomendaciones y Trabajo Futuro

A partir de los resultados obtenidos y las limitaciones observadas durante el desarrollo y la validación, se identifican las siguientes líneas de evolución para el ecosistema PortalAsig.

Soberanía tecnológica mediante modelos locales. PAIMA actualmente delega las fases de Análisis y Planificación a proveedores de inteligencia artificial remotos (APIs compatibles con OpenAI). Como línea de trabajo futuro, se recomienda la integración de modelos de lenguaje ejecutados localmente mediante Ollama, lo que eliminaría la dependencia de conectividad externa y de costos por token, al tiempo que preservaría la confidencialidad de la telemetría operativa dentro de la infraestructura universitaria. Esta transición requeriría ajustar los *prompts* y evaluar la calidad del razonamiento de modelos locales frente a los

proveedores cloud actuales.

Centralización del routing y la autenticación. Se recomienda evaluar la introducción de un gateway —ya sea una extensión de Nginx o un componente dedicado como Spring Cloud Gateway— para simplificar la seguridad perimetral y reducir la complejidad de configuración de CORS en cada microservicio.

Evolución de PAIMA hacia un agente semi-integrado de desarrollo. En su configuración actual, PAIMA opera como un agente de operaciones que monitorea métricas y logs del ecosistema en ejecución. Como línea de evolución, se propone extender su alcance hacia el código fuente mismo, transformándolo en un agente semi-integrado que asista al mantenedor en tareas de ingeniería de software más amplia. Esta evolución contemplaría tres capacidades adicionales.

En primer lugar, la integración con los repositorios de código alojados en GitHub mediante la API oficial del servicio. PAIMA podría leer el historial de *commits*, analizar *pull requests* abiertos, inspeccionar reportes de issues y correlacionar alertas operativas con cambios recientes en el código. Por ejemplo, un pico de latencia en el servicio Site podría asociarse automáticamente con un *merge* reciente que modificó la capa de persistencia, reduciendo el tiempo de diagnóstico desde minutos a segundos.

En segundo lugar, el análisis estático del código fuente combinado con el conocimiento operativo ya acumulado en la base de datos de PAIMA. Mediante la lectura periódica de los repositorios, el agente podría identificar *code smells*, deuda técnica acumulada, inconsistencias en convenciones de nomenclatura y oportunidades de refactorización que escapan a las reglas rígidas de Checkstyle. Esta capacidad trasciende la detección de errores: permite realizar *discovery & improve* sobre el repositorio, sugiriendo extracciones de métodos, simplificaciones de clases complejas o eliminación de dependencias circularizadas que degradan la mantenibilidad del sistema.

En tercer lugar, una interfaz conversacional extendida que permita al administrador no solo consultar sobre incidentes activos, sino también solicitar acciones de mejora continua y asistencia en el desarrollo de nuevas funcionalidades. El administrador podría formular consultas del tipo “¿qué servicio presenta mayor deuda técnica este mes?”, “genera un resumen de los *commits* relacionados con la caída de ayer” o “propón una implementación para el endpoint de importación masiva de semestres que está pendiente”. El agente respondería generando fragmentos de código, diagramas de secuencia o planes de implementación basados en el conocimiento del dominio y del código existente, manteniendo siempre un ciclo de aprobación humana antes de cualquier modificación efectiva.

Esta visión de PAIMA como agente semi-integrado —semi porque opera con supervisión humana, integrado porque conecta operaciones, código y desarrollo en un único ciclo de retroalimentación— representa un paso hacia la autogestión del mantenimiento en su acepción más completa. No se trata de sustituir al ingeniero de software, sino de reducir la fricción entre la observación del sistema, la comprensión de su código y la ejecución de mejoras, especialmente crítico en un contexto donde los colaboradores académicos temporales deben asumir tareas de operación y desarrollo simultáneamente.

Modelos de lenguaje especializados con habilidades operativas. Más

allá del análisis y la recomendación, se propone investigar el entrenamiento de un modelo de lenguaje de pequeña escala (SLM, *Small Language Model*) sobre el dominio académico de PortalAsig, combinado con un sistema de *skills* o herramientas invocables. El objetivo es que el agente no solo converse con el administrador, sino que ejecute directamente operaciones de negocio sobre los microservicios mediante llamadas a sus APIs REST.

Esta arquitectura requeriría definir un catálogo de habilidades estructuradas que cubran las operaciones más frecuentes del sistema: creación de sitios académicos, alta de asignaturas, inscripción masiva de estudiantes mediante archivos CSV, generación de períodos semestrales, configuración de evaluaciones con pesos, y envío de notificaciones institucionales. Cada habilidad se implementaría como una función con esquema de entrada validado (nombre del curso, código, período académico, lista de correos), que el modelo invoca una vez que ha interpretado la intención del usuario a partir de un prompt en lenguaje natural. Por ejemplo, la orden “abre el sitio de Cálculo I para el semestre 2025-1 e inscribe a los estudiantes del listado adjunto” se traduciría en una secuencia de tres llamadas API —creación del sitio, vinculación de la asignatura, carga masiva de participantes— ejecutadas de forma atómica y supervisada.

Para garantizar la seguridad y la trazabilidad, cada invocación de habilidad debería someterse a un ciclo de confirmación humana: el agente presentaría un resumen de las acciones propuestas, los datos afectados y las APIs involucradas, permitiendo al administrador aprobar, modificar o cancelar la operación antes de su ejecución. Este patrón mantiene el principio de autogestión asistida que caracteriza a PAIMA: la inteligencia artificial reduce la carga cognitiva y elimina la necesidad de conocer los endpoints técnicos, pero la autoridad final reside en el operador humano.

El impacto de esta evolución trasciende la comodidad del administrador. En el contexto operativo de PortalAsig, donde los colaboradores académicos temporales deben alternar entre desarrollo de nuevas funcionalidades y operación del sistema en producción, una interfaz conversacional que permita gestionar el dominio académico sin escribir código ni recordar rutas de API desacopla la operación del conocimiento técnico profundo. Un coordinador de asignatura con permisos administrativos podría crear sitios, configurar evaluaciones y enviar notificaciones sin intervención de un ingeniero de software, acelerando los ciclos administrativos del semestre y reduciendo los cuellos de botella que actualmente dependen de la disponibilidad del mantenedor.

Evolución hacia una arquitectura multi-agente distribuida. PAIMA implementa actualmente un ciclo MAPE-K centralizado: un único agente monitorea el ecosistema completo y ejecuta acciones de remediación. Como trabajo futuro de investigación, se propone explorar la evolución hacia una arquitectura tipo AWARE, donde múltiples agentes de inteligencia artificial colaboran entre nodos del ecosistema de forma distribuida [16]. Esta transición representaría un salto desde la autogestión asistida centralizada hacia una computación autónoma plena, donde cada microservicio podría incorporar capacidades de auto-diagnóstico y auto-remediación local, coordinándose con los demás mediante protocolos de consenso multi-agente.

Migración del frontend a Vue.js 3. El frontend actual opera sobre Vue.js 2, versión que alcanzó fin de vida a finales de 2023 y no recibe actualizaciones de seguridad ni correcciones. Como línea de trabajo futuro, se recomienda la migración a Vue.js 3, que introduce la Composition API, una reactividad más eficiente basada en Proxies, mejoras de rendimiento en el renderizado y compatibilidad con herramientas modernas como Vite. Esta migración requeriría actualizar las dependencias principales (Vuex a Pinia o Vue 3 Composition API, Vue Router 4, adaptación de componentes de Bootstrap-Vue) y reescribir los componentes de clase a la sintaxis de `script setup`, pero preservaría la arquitectura modular y los flujos de autenticación ya establecidos.

Cobertura de código y pruebas de integración en el frontend. El backend cuenta con medición sistemática de cobertura mediante JaCoCo, umbral de aceptación en pipeline y pruebas de integración sobre bases de datos reales. El frontend, por su parte, carece de un equivalente: no se instrumenta cobertura de código JavaScript ni se ejecutan pruebas de integración de componentes Vue.js como parte del proceso de validación. Como línea de trabajo futuro, se recomienda incorporar Istanbul/nyc para medir la cobertura de código del frontend, establecer un umbral mínimo en GitHub Actions y complementar las pruebas end-to-end de Cypress con pruebas de integración de componentes mediante Vue Test Utils y Jest. Esta práctica cerraría la brecha de calidad entre la capa de presentación y la capa de aplicación, garantizando que los refactors del store Vuex, los guards del router y los formularios reactivos conserven su comportamiento ante modificaciones.

Alcance explícito fuera del trabajo. Es importante precisar que el presente trabajo se centra en la integración continua (CI) como técnica de aseguramiento de calidad. La entrega continua (CD, *Continuous Delivery*), que automatiza el despliegue de artefactos validados hacia entornos de producción, queda fuera del alcance documentado. La implementación de pipelines de CD requeriría infraestructura adicional de despliegue automatizado sobre los servidores universitarios, lo cual constituye un proyecto independiente de significativa complejidad operativa.

Referencias

- [1] R. A. Vásquez Balza, “Implementación de una metodología de mantenimiento de software para la solución y prevención de fallas del sistema portalesig2,” 2020. Trabajo Especial de Grado, Universidad Central de Venezuela.
- [2] R. E. S. Santos, “Uses, benefits, and limitations of job rotation in software engineering,” *ACM SIGSOFT Software Engineering Notes*, vol. 41, no. 6, pp. 1–4, 2016. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3011286.3011300>.
- [3] T. Berners-Lee, J. Hendler, and O. Lassila, “The semantic web,” *Scientific American*, vol. 284, no. 5, pp. 34–43, 2001.
- [4] G. Edelman, “What is web3, anyway?,” *Wired*, 2021. Interview with Gavin Wood, who coined the term Web 3.0 in a blockchain context in 2014. [Online]. Available: <https://www.wired.com/story/web3-gavin-wood-interview/>.
- [5] M. Shen, Z. Tan, D. Niyato, Y. Liu, J. Kang, Z. Xiong, L. Zhu, W. Wang, and S. Xuemin, “Artificial intelligence for web 3.0: A comprehensive survey,” *arXiv preprint arXiv:2309.09972*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.09972>.
- [6] Z. Kuai, G. Huang, X. Tao, G. Pasi, L. Yao, C. Liu, and N. Zhong, “Web intelligence (WI) 3.0: In search of a better-connected world to create a future intelligent society,” *Artificial Intelligence Review*, 2025.
- [7] C. Richardson, *Microservices patterns: with examples in Java*. Simon and Schuster, 2018.
- [8] B. Foote and J. Yoder, “Big ball of mud,” in *Pattern Languages of Program Design 4*, pp. 654–692, Addison-Wesley, 1997.
- [9] M. Fowler and J. Lewis, “Microservices: a definition of this new architectural term,” *Martin Fowler*, vol. 14, 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>.
- [10] C. Sridharan, *Distributed Systems Observability*. O’Reilly Media, Inc., 2018.

- [11] IEEE, “Iso/iec/ieee international standard for software engineering – software life cycle processes – maintenance,” 2006.
- [12] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *Computer*, vol. 36, no. 1, pp. 41–50, 2003.
- [13] N. C. Mendonça *et al.*, “An architecture for autonomic control of software systems,” *Journal of Systems and Software*, vol. 85, no. 12, pp. 2794–2811, 2012.
- [14] IBM, “An architectural blueprint for autonomic computing,” *IBM White Paper*, 2006.
- [15] M. Esposito, A. Bakhtin, N. Ahmad, M. Robredo, R. Su, V. Lenarduzzi, and D. Taibi, “Autonomic microservice management via agentic AI and MAPE-K integration,” *arXiv preprint arXiv:2506.22185*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.22185>.
- [16] B. P. Sanwouo, C. Quinton, and P. Temple, “Breaking the loop: AWARE is the new MAPE-K,” in *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion ’25)*, (Trondheim, Norway), pp. 626–630, ACM, 2025.
- [17] D. Weyns, *Software Engineering of Self-Adaptive Systems*. Springer, 2019.
- [18] P. Notaro *et al.*, “A survey of aiops methods for failure management,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 4, pp. 1–45, 2021.
- [19] S. Newman, *Building microservices: designing fine-grained systems*. O’Reilly Media, Inc., 2015.
- [20] R. Gate, “Flyway documentation.” <https://documentation.red-gate.com/flyway>, 2024. Accessed: 2026-05-21.
- [21] AtomicJar, “Testcontainers.” <https://testcontainers.com/>, 2024. Accessed: 2026-05-21.
- [22] Cypress.io, “Cypress documentation.” <https://docs.cypress.io/>, 2024. Accessed: 2026-05-21.
- [23] P. Authors, “Prometheus documentation.” <https://prometheus.io/docs/introduction/overview/>, 2024. Accessed: 2026-05-21.
- [24] G. Labs, “Grafana documentation.” <https://grafana.com/docs/>, 2024. Accessed: 2026-05-21.
- [25] D. Inc., “Docker compose documentation.” <https://docs.docker.com/compose/>, 2024. Accessed: 2026-05-21.

- [26] D. Hardt, “The oauth 2.0 authorization framework.” RFC 6749, IETF, 2012. [Online]. Available: <https://tools.ietf.org/html/rfc6749>.
- [27] M. Jones, J. Bradley, and N. Sakimura, “Json web token (JWT).” RFC 7519, IETF, 2015. [Online]. Available: <https://tools.ietf.org/html/rfc7519>.
- [28] B. Gregg *et al.*, “Chaos engineering.” Netflix Tech Blog, 2011. [Online]. Available: <https://netflixtechblog.com/tagged/chaos-engineering>.